BACHELOR OF SCIENCE IN COMPUTER SCIENCE AND ENGINEERING

Evaluating Efficiency and Robustness of Variable Timing in Spiking Neural Networks

Nayeem Fardin Rohan
210041141
Md. Ifti Akkas
210041111
Ahsanul Amin Anik
210041126

Department of Computer Science and Engineering
Islamic University of Technology
September, 2026

Evaluating Efficiency and Robustness of Variable Timing in Spiking Neural Networks

Nayeem Fardin Rohan
210041141
Md. Ifti Akkas
210041111
Ahsanul Amin Anik
210041126

Department of Computer Science and Engineering
Islamic University of Technology
September, 2026

Declaration of Candidate

This is to certify that the work presented in this thesis is the outcome of the analysis and experiments carried out by **Nayeem Fardin Rohan**, **Md. Ifti Akkas**, and **Ahsanul Amin Anik** under the supervision of **Aashnan Rahman**, Junior Lecturer, Department of Computer Science and Engineering, Islamic University of Technology, Dhaka, Bangladesh. It is also declared that neither this thesis nor any part of it has been submitted anywhere else for any degree or diploma. Information derived from the published and unpublished work of others have been acknowledged in the text and a list of references is given.

Aashnan Rahman
Junior Lecturer
Department of Computer Science and Engineering
Islamic University of Technology (IUT)
Date: September 18, 2026

Nayeem Fardin Rohan
Student ID: 210041141
Date: September 18, 2026

Md. Ifti Akkas
Student ID: 210041111
Date: September 18, 2026

Ahsanul Amin Anik
Student ID: 210041126
Date: September 18, 2026

Dedicated to our families, for their patience through every late night of this work

Contents

1 Introduction **1**

1.1 Background . . . 2
1.1.1 Spiking neural networks and the timestep budget . . . 2
1.1.2 Variable timing . . . 2
1.1.3 Adversarial robustness of SNNs . . . 3
1.2 Motivation . . . 3
1.3 Problem Statement . . . 4
1.4 Research Questions . . . 4
1.5 Objectives . . . 5
1.6 Research Challenges . . . 5
1.7 Contributions . . . 6
1.8 Scope and Delimitations . . . 8
1.9 Thesis Organisation . . . 8
1.10 Summary . . . 9

2 Related Works **10**

2.1 Spiking Neural Networks . . . 10
2.1.1 From biological neurons to the LIF model . . . 10
2.1.2 Training spiking neural networks . . . 11
2.1.3 Measuring the cost of SNN inference . . . 12
2.2 Adaptive and Early-Exit Inference . . . 12
2.2.1 Adaptive computation in conventional networks . . . 12
2.2.2 Confidence measures for exit decisions . . . 13
2.2.3 Variable timing in spiking neural networks . . . 13
2.3 Adversarial Robustness . . . 14
2.3.1 Adversarial examples and gradient-based attacks . . . 14
2.3.2 Adversarial training . . . 14
2.3.3 Pitfalls of robustness evaluation . . . 15

2.4 Adversarial Robustness of Spiking Neural Networks 16
2.4.1 Early claims of inherent robustness 16
2.4.2 Robust training and certification 16
2.4.3 Reliable evaluation of SNN robustness 16
2.4.4 What is missing 17
2.5 Attacks on the Computation of Adaptive Networks 17
2.6 Variance and Reporting Practice 17
2.7 Synthesis and Research Gap 18
2.8 Summary 19

3 Methodology **20**
3.1 Overview 20
3.2 Problem Formulation 21
3.2.1 Spiking neurons 21
3.2.2 Input encoding and readout 22
3.2.3 Fixed-timestep inference 22
3.2.4 Variable timing 22
3.3 Backbone Architectures 23
3.4 Training 24
3.4.1 Training regimes 24
3.4.2 Timestep schedules 25
3.4.3 Optimisation and hyperparameters 25
3.4.4 Model configurations 25
3.5 Data and Splits 25
3.6 Compute Accounting 26
3.7 Compute-Matched Evaluation Protocol 27
3.8 Operating-Point Selection 28
3.9 Stopping Rules 29
3.10 Threat Model and Attacks 30
3.10.1 Threat model 30
3.10.2 Attacking through the exit gate 30
3.10.3 Misclassification attacks 30
3.10.4 Gate-specific attacks 31
3.10.5 Attack variance 31
3.11 Multi-Seed Replication and Statistical Analysis 32
3.11.1 Multi-seed design 32
3.11.2 Paired differences 32
3.11.3 Tests 32

3.12 Implementation and Reproducibility . . . 33
3.13 Summary . . . 34

4 Results and Discussion **35**
4.1 Mapping Objectives to Experiments . . . 35
4.2 Setup and Metrics . . . 35
4.3 Fixed-Timestep Behaviour . . . 36
4.4 RQ1: Efficiency at Equal Compute . . . 38
4.4.1 The compute–accuracy frontier . . . 38
4.4.2 How large is the gain, and is it real? . . . 38
4.4.3 Where do inputs exit? . . . 41
4.4.4 Effect of the training schedule . . . 42
4.5 Stopping Rules . . . 43
4.6 RQ4: Sensitivity to the Selection Rule . . . 44
4.6.1 Max-gain selection is biased . . . 44
4.6.2 Sensitivity to the tolerance . . . 46
4.7 RQ2: Adversarial Robustness . . . 47
4.7.1 Clean-trained models have no robustness to lose . . . 47
4.7.2 PGD and AutoAttack disagree . . . 48
4.7.3 What attacks do to exit depth . . . 49
4.8 RQ3: The Exit Gate as an Attack Surface . . . 49
4.8.1 Wrong early exit . . . 49
4.8.2 Compute overuse . . . 50
4.8.3 Summary of the attack surface . . . 52
4.9 Multi-Seed Replication . . . 52
4.9.1 Per-seed results . . . 52
4.9.2 Paired tests . . . 53
4.9.3 Interpretation: robustness follows compute . . . 55
4.9.4 Attack-seed variance . . . 55
4.10 Discussion . . . 55
4.10.1 What VT is, and what it is not . . . 55
4.10.2 Why PGD-only evaluation would have misled us . . . 56
4.10.3 Compute as an attack surface . . . 56
4.10.4 Recommendations for evaluating VT . . . 56
4.11 Threats to Validity . . . 57
4.11.1 Internal validity . . . 57
4.11.2 External validity . . . 58
4.11.3 Construct validity . . . 58

4.12 Summary 59

5 Conclusion **60**
5.1 Restatement of the Problem 60
5.2 Answers to the Research Questions 60
5.3 Objectives Revisited 62
5.4 Summary of Contributions 63
5.5 Implications 63
5.6 Limitations 64
5.7 Future Work 65
5.8 Concluding Remarks 66

References **67**

Appendices **74**

A Hyperparameters **75**

B Additional Results **77**

C Reproducibility **85**
C.1 Repository Layout 85
C.2 Regenerating the Results 86
C.3 Determinism 86

List of Figures

3.1 Overview of the experimental pipeline 21
3.2 Variable-timing inference 23
3.3 Data splits 26

4.1 Fixed-timestep accuracy as a function of the horizon 37
4.2 Compute–accuracy frontier for the clean-trained ResNet-18 39
4.3 Compute–accuracy frontiers for all configurations 40
4.4 Distribution of exit timesteps at the VT-neutral operating points 41
4.5 Effect of the training schedule on the ResNet-18 frontiers 42
4.6 Stopping rules compared at matched exit depth 44
4.7 Effect of the selection rule on the reported gain 45
4.8 Sensitivity of the VT-neutral gain to the tolerance δ. 47
4.9 VT versus the full horizon under PGD-20 and AutoAttack 48
4.10 Accuracy and confidence of the running prediction over timesteps . . 50
4.11 Exit-depth inflation under the compute-overuse attack 51
4.12 Paired multi-seed differences 54

B.1 Training curves of the multi-seed adversarially trained models 77

List of Tables

1.1 Mapping of contributions to research questions, objectives and chapters. 7

2.1 Positioning of this thesis against the closest prior work. 19

3.1 Backbones used in this thesis. Dense operations are the multiply–accumulate operations a non-spiking network of the same shape would perform over eight timesteps, per image. 24
3.2 Model configurations. The short names are used in the results tables and figures. 26

4.1 Experiments, the research questions they address, and where their results are reported. 36
4.2 Clean test accuracy (%) of fixed-timestep inference at each horizon T ($n = 10{,}000$). 37
4.3 VT-neutral operating points 39
4.4 Stopping-rule comparison 43
4.5 Max-gain versus VT-neutral operating points 45
4.6 Gain over the compute-matched control (points) under VT-neutral selection with tolerance δ. 46
4.7 Single-run robustness of fixed and VT inference 48
4.8 Wrong-early-exit attack versus untargeted PGD 50
4.9 Compute-overuse attack 51
4.10 Multi-seed results per checkpoint 53
4.11 Pooled paired differences across six adversarially trained checkpoints 53

5.1 Summary of the answers to the research questions. 61

A.1 Training, model and evaluation hyperparameters. 76

B.13 All single-run attack results 77
B.1 SynOps of fixed-timestep inference 81

B.2 VT sweep for R18 clean (test set, max-softmax rule). 81
B.3 VT sweep for R18 adv (1) (test set, max-softmax rule). 81
B.4 VT sweep for R18 adv (2) (test set, max-softmax rule). 81
B.5 VT sweep for R18 clean mixT (test set, max-softmax rule). 82
B.6 VT sweep for R18 adv mixT (test set, max-softmax rule). 82
B.7 VT sweep for VGG clean (test set, max-softmax rule). 82
B.8 VT sweep for VGG adv (test set, max-softmax rule). 82
B.9 VT sweep for MBv2 clean (test set, max-softmax rule). 83
B.10 VT sweep for MBv2 adv (test set, max-softmax rule). 83
B.11 Compute at the VT-neutral operating points 83
B.12 Stopping rules at degenerate reference depths 84
B.14 Attack-seed variance 84
B.15 Multi-seed training summaries 84

List of Abbreviations

AA	AutoAttack
ACT	Adaptive Computation Time
AMP	Automatic Mixed Precision
ANN	Artificial Neural Network
APGD	Auto-Projected Gradient Descent
BN	Batch Normalisation
BPTT	Backpropagation Through Time
CE	Cross-Entropy
CI	Confidence Interval
EoT	Expectation over Transformation
FAB	Fast Adaptive Boundary attack
FGSM	Fast Gradient Sign Method
LIF	Leaky Integrate-and-Fire
MAC	Multiply–Accumulate operation
MSP	Maximum Softmax Probability
PGD	Projected Gradient Descent
RFGSM	FGSM with Random Initialisation
RQ	Research Question
SGD	Stochastic Gradient Descent
SNN	Spiking Neural Network
SynOps	Synaptic Operations
VT	Variable Timing

Acknowledgement

We are deeply grateful to our supervisor, Aashnan Rahman, for guidance throughout this thesis. The insistence that every claim be backed by a controlled comparison shaped the methodology of this work more than any other single influence, and patience when results refused to behave as expected kept the project moving.

We thank the faculty members of the Department of Computer Science and Engineering at the Islamic University of Technology for the education that made this work possible, and the members of our pre-defence and defence committees, whose questions sharpened both the experiments and the way we report them.

We acknowledge the authors of the open-source software on which this work rests, in particular the BrainCog / RandHet-SNN code base from which our experimental suite descends, PyTorch, and the AutoAttack and torchattacks libraries. We also thank Kaggle for the GPU time that made multi-seed adversarial training feasible for an undergraduate project.

Finally, we thank our families and friends for their support and understanding throughout.

Abstract

Spiking neural networks (SNNs) process an input over several discrete timesteps, so their inference cost grows with the number of timesteps they are allowed to run. *Variable timing* (VT) lets each input stop as soon as the network's accumulated prediction is confident enough, promising lower average compute at little cost in accuracy. Two questions are usually left open: whether the reported savings survive a fair comparison against a fixed-horizon SNN that uses the same compute, and what the confidence-gated stopping rule does to the network's behaviour under attack.

This thesis studies both questions on CIFAR-10 with directly trained, deterministic leaky integrate-and-fire SNNs built on ResNet-18, VGG16 and MobileNetV2 backbones, trained either on clean data or entirely on adversarial examples, with a fixed eight-step or a mixed one-to-eight-step training schedule. Every VT operating point is compared with the fixed-timestep model that matches its measured synaptic operations, and thresholds are chosen by a selection rule that does not favour VT.

At equal compute, VT is more accurate than fixed-timestep inference on all nine configurations, by +0.14 to +3.37 percentage points, and the clean-data advantage replicates across three independent training seeds for each of two adversarially trained backbones ($+2.47 \pm 0.99$ points, $t(5) = 6.11$, $p = 0.002$). Under attack, the picture separates by attack strength. Against PGD-20, VT appears to be as robust as the full eight-step model. Against AutoAttack, VT is consistently *less* robust than the full-horizon model (-4.05 ± 2.88 points, 0 of 6 seeds positive, $p = 0.018$), yet indistinguishable from the compute-matched fixed-timestep model ($+0.35 \pm 4.33$, $p = 0.85$): the robustness VT gives up is the robustness that the skipped timesteps would have provided. We also introduce a label-preserving compute-overuse attack that targets the exit gate itself: it raises the average exit depth of clean-trained, fixed-schedule VT models by more than one full timestep while leaving most predictions correct, an energy-latency attack surface that fixed-horizon inference does not have. Adversarial training almost entirely removes it.

Operating points are selected with a VT-neutral rule, because maximising the gain over the matched baseline can select degenerate thresholds far below full-horizon accuracy. The results argue that VT should be reported as a compute-accuracy trade-off with an explicit attack model, evaluated with strong attacks, matched controls and multiple seeds.

Chapter 1

Introduction

Deep neural networks have become the default tool for perception tasks, but their cost has grown with them. A modern image classifier performs billions of multiply–accumulate operations per input, and the energy those operations consume is now a first-order concern both in data centres and on battery-powered devices [28]. *Spiking neural networks* (SNNs) are one of the most studied responses to this problem. Instead of passing real-valued activations between layers once, an SNN passes binary *spikes* over a number of discrete timesteps, and a synaptic operation is performed only when a spike actually arrives [41], [52]. On neuromorphic hardware such as TrueNorth and Loihi [12], [44], that sparsity translates directly into energy savings.

The same mechanism that makes SNNs efficient also creates a knob that conventional networks do not have: the number of timesteps. Running an SNN for more timesteps lets its neurons integrate more evidence and usually improves accuracy, but every extra timestep costs another pass of synaptic activity. Most SNNs fix this number in advance for every input. *Variable timing* (VT), also called dynamic timesteps or delay-adaptive inference, removes that restriction: the network keeps a running prediction, and each input stops as soon as that prediction is confident enough [7], [36], [37]. Easy inputs exit after a few timesteps, hard inputs use the full budget, and the average cost falls.

This thesis is about what VT really buys and what it costs. It asks whether the efficiency gain survives a fair, compute-matched comparison, and what a confidence-gated stopping rule does to an SNN when an adversary is present. The answers turn out to be more nuanced than either the efficiency literature or the robustness literature, read on their own, would suggest.

1.1 Background

1.1.1 Spiking neural networks and the timestep budget

The basic unit of most practical SNNs is the leaky integrate-and-fire (LIF) neuron [18]. At each timestep the neuron adds its weighted input current to a membrane potential that slowly leaks towards zero. When the potential crosses a threshold, the neuron emits a spike and its potential is reset. A network of LIF neurons therefore turns a static input into a spatio-temporal pattern of spikes, and the network's prediction is read out by accumulating the output layer's activity over the timesteps that were run.

Training such networks directly was long considered impractical because the spike function has no useful gradient. Surrogate-gradient methods [47], [70] replace that gradient with a smooth approximation during backpropagation through time, and together with normalisation schemes such as threshold-dependent batch normalisation [72] and residual designs such as SEW-ResNet [17] they now let deep SNNs be trained end-to-end on standard vision benchmarks with a handful of timesteps. The alternative route, converting a trained artificial network into an SNN [53], typically needs many more timesteps to reach the same accuracy.

In both cases the number of timesteps T sits at the centre of the efficiency story. Synaptic operations, the additions triggered by incoming spikes, grow roughly linearly with T, and so does latency. Whether an SNN is actually more energy-efficient than an equivalent artificial network depends heavily on how many timesteps it needs and how sparse its activity is [11], [60]. Reducing the average number of timesteps is therefore one of the most direct ways to make SNNs cheaper.

1.1.2 Variable timing

Variable timing applies the idea of early-exit networks [29], [31], [35], [62] along the time axis rather than the depth axis. Because an SNN already produces a prediction after every timestep, no extra classifier heads are needed: after timestep t the network computes a confidence score from its accumulated output, and if the score exceeds a threshold θ the input exits with that prediction. Lower thresholds exit earlier and save more compute, while higher thresholds behave more like a fixed-horizon network. Sweeping θ traces out a curve of accuracy against average compute.

Several recent works report that this simple mechanism recovers most of the accuracy of a long fixed horizon at a fraction of the cost [36], [37], and others add statistical guarantees on the reliability of the stopping decision [7] or adapt it to medical

imaging [50].

1.1.3 Adversarial robustness of SNNs

Neural networks are vulnerable to *adversarial examples*: inputs changed by a small, carefully chosen perturbation that flips the prediction [19], [61]. The standard defence is adversarial training, in which the model is trained on such perturbed inputs [42]. SNNs were initially reported to be more robust than artificial networks [56], [57], but much of that apparent robustness came from the difficulty of computing gradients through spikes rather than from genuine resistance, and later work has shown that SNN robustness must be measured with attacks that are adapted to their non-differentiable dynamics [38], [66]. The adversarial-machine-learning community has learnt the same lesson repeatedly: weak attacks and gradient masking produce robustness numbers that do not survive a stronger evaluation [2], [10], [63].

1.2 Motivation

VT and robustness have so far been studied largely in isolation. The efficiency literature evaluates VT on clean data, and the SNN robustness literature evaluates fixed-horizon networks. Putting the two together raises questions that neither literature answers on its own.

Is the comparison fair? VT is usually shown as a curve of accuracy against average timesteps, next to the accuracy of a fixed-horizon network at its full budget. That comparison mixes two effects: VT spends less compute, and it chooses *where* to spend it. The fair question is whether VT beats a fixed-horizon network that uses the *same* compute. Without that compute-matched control, an efficiency claim cannot be separated from the trivial observation that running fewer timesteps is cheaper.

Does early exit change robustness? If an adversary can make an input exit early with a wrong but confident prediction, VT might be easier to attack than a fixed-horizon network. Conversely, if robust features take several timesteps to build up, the robustness of a VT model might simply track how many timesteps it ends up running. Which of these dominates, and whether the answer depends on how the network was trained, is not known.

Does the gate create a new attack surface? An input-dependent exit means that compute itself becomes input-dependent. Work on multi-exit artificial networks has shown that adversaries can craft inputs that defeat early exits and force maximum computation [23], [27], a family of attacks closely related to sponge examples [58]. For an SNN whose selling point is energy efficiency, an attack that silently removes that efficiency is a security problem in its own right.

Are the conclusions reliable? Each of these comparisons depends on choices that can change the conclusion: how the threshold is selected, which attack is used and how many training seeds are run. Deep learning results are known to be sensitive to random seeds and evaluation protocol [4], [25], [49], and adversarial robustness results to the strength of the attack [5], [8]. A study of VT under attack is only useful if its conclusions survive these sensitivities.

1.3 Problem Statement

Variable timing promises to reduce the inference cost of spiking neural networks by letting each input stop when the network is confident. Existing evaluations compare VT with fixed-horizon SNNs at unequal compute, choose operating points in ways that can favour VT, and measure accuracy only on clean data. As a result it is not known (i) whether VT is more accurate than a fixed-horizon SNN that uses the same compute, (ii) how the confidence gate changes adversarial robustness and whether it opens new compute-targeting attacks, and (iii) whether these conclusions hold across architectures, training regimes, training seeds, attack strengths and operating-point selection rules.

1.4 Research Questions

The thesis is organised around four research questions.

RQ1 (Efficiency)

At equal measured compute, is a VT SNN more accurate than a fixed-timestep SNN, and does the answer hold across backbones, training regimes, training schedules and training seeds?

RQ2 (Robustness)

How does VT change adversarial robustness relative to (a) the same network

run for its full horizon and (b) the compute-matched fixed-timestep network, and does the answer depend on the strength of the attack?

RQ3 (Attack surface)
Does the confidence gate create attack objectives that fixed-horizon inference does not have, in particular attacks that inflate compute while preserving the label, and how does the training regime affect them?

RQ4 (Evaluation reliability)
How sensitive are the answers to RQ1–RQ3 to the operating-point selection rule, the attack used, and run-to-run variance?

1.5 Objectives

To answer these questions, the thesis sets the following objectives.

- **O1.** Build a reproducible evaluation suite in which every VT operating point is compared with the fixed-timestep network that matches its measured synaptic operations, with sample-exact compute accounting.
- **O2.** Train deterministic LIF SNNs on three backbones (ResNet-18, VGG16, MobileNetV2) under two training regimes (fully clean, fully adversarial) and two timestep schedules (fixed eight-step, mixed one-to-eight-step), and measure VT's compute–accuracy frontier on each.
- **O3.** Evaluate robustness of fixed-horizon, compute-matched and VT inference under PGD-20 and AutoAttack, with the attacker differentiating through the exit gate.
- **O4.** Implement and evaluate two gate-specific attacks: a *wrong-early-exit* attack and a label-preserving *compute-overuse* attack.
- **O5.** Quantify the reliability of the conclusions through a VT-neutral operating-point selection rule, a tolerance sensitivity analysis, alternative stopping rules, and replication across three training seeds with paired statistical tests.

1.6 Research Challenges

Several practical difficulties shaped the design of this study.

Measuring compute fairly. The cost of a VT model depends on which inputs exit when. Measuring it with batch-level averages over-counts compute, because the slowest input in a batch keeps the whole batch running. We therefore implemented sample-exact accounting, in which each input's synaptic operations are counted only up to its own exit step (Section 3.6).

Attacking a non-differentiable gate. The exit decision is a hard threshold, so gradients do not flow through it. An attacker who ignores the gate may under-estimate VT's vulnerability, and one who differentiates only through the full-horizon output attacks a different model. We follow the adaptive-attack principle [63] and give the attacker the full trace of per-timestep logits, with the loss computed at each input's actual exit step (Section 3.10).

Compute constraints. Adversarial training of SNNs is expensive: every training step requires a full multi-timestep forward and backward pass to generate the perturbation and another to update the weights. All experiments were run on Kaggle GPUs under session time limits, which bounded the number of training seeds we could afford (three per adversarially trained backbone) and forced AutoAttack to be run on a 2,000-image class-balanced subset of the test set. One MobileNetV2 multi-seed run exceeded the 12-hour session limit and produced only partial results.

Selection bias. Every VT result is reported at some threshold, and the threshold must be chosen. We found that the natural choice, the threshold with the largest gain over its compute-matched control, can select operating points that are far below the full-horizon accuracy, and reported gains that are artefacts of comparing two poor models (Section 4.6). Avoiding this required a selection rule that does not reward VT for the comparison it is being tested on.

Variance. Our first adversarially trained ResNet-18 and a later re-run with identical settings differed by 3.5 points in full-horizon accuracy and by a factor of two in VT's gain. Single-run results were therefore not enough, and the robustness conclusions in particular had to be re-examined with multiple training seeds.

1.7 Contributions

The main contributions of this thesis are:

C1. A compute-matched evaluation protocol for VT. We compare every VT operating point with the fixed-timestep model of nearest measured synaptic operations, using sample-exact compute accounting, and select operating points on a held-out calibration split with a VT-neutral rule that does not reward VT for the comparison being made.

C2. A label-preserving compute-overuse attack on VT SNNs. We adapt slowdown attacks on multi-exit networks to the time axis of SNNs: an attacker who differentiates through the full trace of per-timestep outputs suppresses the exit confidence while preserving the correct label. It raises the mean exit depth of clean-trained fixed-schedule VT models by 1.15–1.19 timesteps with predictions left largely intact, an energy-latency attack surface that fixed-horizon inference does not have. Adversarial training reduces the inflation to 0.04–0.37 timesteps.

C3. Evidence that VT's clean efficiency gain is real. At equal compute, VT is more accurate than fixed-timestep inference on all nine model configurations we trained (gains of +0.14 to +3.37 points), and the gain replicates across three training seeds for two adversarially trained backbones ($+2.47 \pm 0.99$ points, $p = 0.002$).

C4. A decomposition of VT's robustness cost. Under AutoAttack, VT is consistently less robust than the full-horizon model (-4.05 points, 0 of 6 seeds positive) but indistinguishable from the compute-matched model (+0.35 points, $p = 0.85$). The robustness VT loses is the robustness its skipped timesteps would have provided. PGD-20 hides this cost entirely.

C5. An open, reproducible suite of Kaggle-ready notebooks, raw result files and analysis scripts from which every number in this thesis is regenerated.

Table 1.1 maps the contributions to the research questions and to the chapters in which they are developed.

Table 1.1: Mapping of contributions to research questions, objectives and chapters.

Contribution	Research question	Objectives	Where
C1 Compute-matched protocol	RQ1, RQ4	O1, O5	Section 3.7, Section 3.8
C2 Compute-overuse attack	RQ3	O4	Section 3.10, Section 4.8
C3 Clean efficiency gain	RQ1	O2	Section 4.4, Section 4.9
C4 Robustness decomposition	RQ2	O3	Section 4.7, Section 4.9
C5 Reproducible suite	—	O1	Section 3.12, Appendix C

1.8 Scope and Delimitations

To keep the study controlled, we deliberately limit its scope.

- **Dataset.** All experiments use CIFAR-10 [33]. Static images with direct encoding are the setting in which most VT results have been reported. Neuromorphic event datasets are left for future work.
- **Neuron model.** We use deterministic LIF neurons with a fixed threshold and leak. Stochastic or heterogeneous neurons, which have been proposed as robustness mechanisms in their own right [67], are excluded so that any robustness effect can be attributed to VT rather than to randomness.
- **Threat model.** We consider white-box ℓ_∞ attacks with $\epsilon = 8/255$ on the input image. Physical-world attacks, hardware fault injection and attacks on the neuromorphic substrate are outside scope.
- **Compute metric.** We measure synaptic operations (SynOps), a hardware-independent proxy for energy. We do not measure energy on neuromorphic hardware, since the relationship between SynOps and joules depends on the platform [11].
- **Horizon.** The maximum horizon is $T = 8$ timesteps throughout, a common choice for directly trained SNNs on CIFAR-10.

1.9 Thesis Organisation

The rest of the thesis is organised as follows. Chapter 2 reviews work on SNN training and efficiency, adaptive and early-exit inference, adversarial robustness of SNNs, attacks on adaptive inference, and evaluation methodology, and positions this thesis against it. Chapter 3 formalises VT, describes the models, training regimes, compute accounting, operating-point selection, attacks and statistical tests, and documents the implementation. Chapter 4 presents the experiments: the compute-matched efficiency comparison, the effect of the selection rule, the robustness comparison, the gate-specific attacks, the multi-seed replication and the threats to validity. Chapter 5 answers the research questions, discusses implications and limitations, and outlines future work. The appendices contain full result tables, hyperparameters and reproducibility instructions.

1.10 Summary

Variable timing lets an SNN spend less compute on easy inputs. This thesis asks whether that saving is real when compared fairly, what it does to adversarial robustness, and whether the exit gate can itself be attacked. The next chapter situates these questions in the literature.

Chapter 2

Related Works

This chapter reviews the five bodies of work on which the thesis draws. Section 2.1 covers spiking neural networks: neuron models, training methods, and how their energy cost is measured. Section 2.2 covers adaptive and early-exit inference, first in conventional networks and then in SNNs, where it takes the form of variable timing. Section 2.3 reviews adversarial attacks, adversarial training and the pitfalls of robustness evaluation, and Section 2.4 narrows this to SNNs. Section 2.5 reviews attacks that target the computation of adaptive networks rather than their predictions, and Section 2.6 reviews work on variance and reporting practice in machine learning. Section 2.7 synthesises the gaps that motivate this thesis and positions it against the closest prior work.

2.1 Spiking Neural Networks

2.1.1 From biological neurons to the LIF model

Maass [41] framed networks of spiking neurons as the "third generation" of neural network models, after threshold units and sigmoidal units, and showed that they are at least as computationally powerful as their predecessors. In a spiking network, information is carried by the timing and number of discrete events rather than by real-valued activations. Neuroscience offers a hierarchy of neuron models of this kind, from detailed conductance-based models to the simple leaky integrate-and-fire (LIF) model [18]. The LIF neuron keeps a single state variable, its membrane potential, which integrates input current, decays towards a resting value with time constant τ, and triggers a spike and a reset whenever it crosses a threshold. Its simplicity has made it the default choice for machine-learning SNNs, and it is the model used throughout

this thesis.

Roy, Jaiswal and Panda [52] survey the case for spike-based machine intelligence: by computing only when spikes arrive, SNNs replace dense multiply–accumulate operations with sparse, event-driven accumulations, which is well suited to neuromorphic hardware. Two influential chips illustrate this. IBM's TrueNorth [44] integrates a million spiking neurons with an event-driven communication network, and Intel's Loihi [12] adds on-chip learning to a many-core neuromorphic design. On such hardware, the energy of inference is dominated by the number of synaptic events, which is why both spike sparsity and the number of timesteps matter.

2.1.2 Training spiking neural networks

The spike function is a step: its derivative is zero almost everywhere and undefined at the threshold. This makes gradient-based training of SNNs non-trivial, and two families of methods have emerged.

ANN-to-SNN conversion. Conversion methods train a conventional artificial network with ReLU activations and then map its weights onto an SNN whose firing rates approximate the original activations [53]. Conversion inherits the accuracy of mature ANN training pipelines, but rate approximation becomes accurate only over many timesteps, often hundreds, which undermines the efficiency argument.

Direct training with surrogate gradients. The alternative is to train the SNN itself with backpropagation through time (BPTT), replacing the derivative of the spike function by a smooth surrogate in the backward pass [47]. Spatio-temporal backpropagation [70] made this practical for deep networks, and subsequent work removed further obstacles. Threshold-dependent batch normalisation [72] stabilises the scale of membrane inputs in deep SNNs. Spike-element-wise residual networks [17] adapt residual learning [24] to spiking dynamics, and temporal efficient training [13] reweights the loss across timesteps so that the network is useful at every step rather than only at the last one. Architecture search has been used to find SNN topologies that are both accurate and sparse [46]. Directly trained SNNs now reach competitive accuracy on CIFAR-10 with as few as four to eight timesteps.

The last point, how a network behaves at intermediate timesteps, matters directly for variable timing. A network trained only with its final-timestep output supervised may produce poor predictions at early timesteps, so early exit would be costly. Training with a randomly sampled horizon, or with a loss applied at every timestep, encourages

useful early predictions. This thesis studies one such intervention, a mixed-timestep schedule in which the training horizon is sampled uniformly from one to eight for each batch (Section 3.4).

2.1.3 Measuring the cost of SNN inference

Comparing the efficiency of SNNs requires a cost model. The most common hardware-independent proxy is the number of synaptic operations (SynOps): for each layer, the number of accumulate operations triggered by incoming non-zero activations. Horowitz [28] estimated that, in a 45 nm process, a 32-bit floating-point addition costs roughly a fifth of the energy of a multiplication, and such figures are widely used to convert SynOps and dense multiply–accumulate counts into energy estimates.

These estimates have been criticised. Dampfhoffer et al. [11] show with a hardware-aware analysis that SNNs are more energy-efficient than equivalent ANNs only when spike activity is low and the number of timesteps small, because memory access and membrane-state updates add costs that SynOps ignore. Sorbaro et al. [60] show that energy-aware training can substantially reduce spike counts. The common conclusion is that the number of timesteps is one of the dominant terms in SNN cost. This thesis uses SynOps as its compute metric and, crucially, uses it to *match* VT against fixed-timestep inference rather than to make absolute energy claims (Section 3.6).

2.2 Adaptive and Early-Exit Inference

2.2.1 Adaptive computation in conventional networks

The idea that a network should spend more computation on harder inputs predates SNNs. Adaptive Computation Time [20] lets a recurrent network learn how many internal steps to take per input, with a halting unit and a ponder cost that penalises extra steps. In feed-forward networks, the same idea takes the form of early exits. BranchyNet [62] attaches side classifiers to intermediate layers and exits when the entropy of a side classifier's output is low enough. Multi-Scale Dense Networks [29] redesign the architecture so that early classifiers have access to coarse-scale features, enabling anytime and budgeted-batch prediction. Shallow-Deep Networks [31] show that many inputs are classified correctly by early layers and that later layers can even "overthink" and change a correct early prediction into a wrong one.

Han et al. [22] survey dynamic neural networks more broadly, distinguishing instance-wise, spatial-wise and temporal-wise adaptation, and Laskaridis, Kouris and Lane [35]

survey early-exit networks specifically, highlighting the design of exit policies, the training of multi-exit models, and deployment challenges. Both surveys note that early-exit methods are usually evaluated by plotting accuracy against average cost and comparing with the full model, a protocol whose limitations we return to below.

2.2.2 Confidence measures for exit decisions

An exit policy needs a score that predicts whether the current prediction is correct. The simplest is the maximum softmax probability (MSP), which Hendrycks and Gimpel [26] established as a strong baseline for detecting misclassified and out-of-distribution inputs. Alternatives include the margin between the two largest probabilities, the normalised entropy of the predictive distribution, temperature-scaled softmax (temperature scaling is the standard post-hoc remedy for the over-confidence of modern networks [21]), and evidential scores that interpret outputs as the parameters of a Dirichlet distribution [55]. All of these are functions of the network's output only, which makes them cheap but also means that they can be manipulated by anyone who can manipulate the output. This thesis compares five such scores as stopping rules (Section 3.9).

2.2.3 Variable timing in spiking neural networks

SNNs are naturally suited to early exit along the time axis. Because the output layer's activity is accumulated over timesteps, a prediction is available after every timestep at no extra cost, and stopping early saves every subsequent timestep of synaptic activity across the whole network.

Dynamic Confidence. Li, Jones and Furber [36] propose using the confidence of the SNN's accumulated output to decide, at each timestep, whether to terminate inference. Applied to converted SNNs, the approach substantially reduces the average number of timesteps needed to reach a given accuracy, demonstrating that much of the latency of rate-coded SNNs is spent on inputs that were already classified correctly.

Dynamic-timestep SNNs. Li, Moitra, Geller and Panda [37] propose input-aware dynamic timestep SNNs (DT-SNN), which compute the entropy of the accumulated output after each timestep and stop when it falls below a threshold. They evaluate the method on in-memory computing hardware and report large improvements in energy–delay product with little accuracy loss.

Delay-adaptive SNNs with guarantees. Chen, Park and Simeone [7] approach the stopping decision from the viewpoint of reliability. Their SpikeCP method uses conformal prediction to produce set-valued predictions at each timestep and stops when the set is small enough, providing a guarantee on the coverage of the final decision regardless of when the model stops.

Uncertainty-based variable timesteps. The VT-SNN of Rao et al. [50] adapts the number of timesteps per input using an uncertainty measure and applies it to brain disease diagnosis, showing that the idea transfers beyond standard vision benchmarks.

These works establish that VT can reduce SNN latency and energy at small cost in clean accuracy. They differ in the confidence score, the training procedure and the hardware, but they share two features that this thesis revisits. First, their baselines are fixed-horizon SNNs evaluated at their full horizon or at a few chosen horizons, rather than at the horizon that matches VT's measured compute. Second, they evaluate on clean data, and none studies how the exit decision behaves under adversarial perturbation.

2.3 Adversarial Robustness

2.3.1 Adversarial examples and gradient-based attacks

Szegedy et al. [61] discovered that imperceptible perturbations can change the prediction of an image classifier, and Goodfellow, Shlens and Szegedy [19] explained the phenomenon by the locally linear behaviour of neural networks, introducing the Fast Gradient Sign Method (FGSM), which perturbs each input pixel by ϵ in the direction of the sign of the loss gradient. Madry et al. [42] framed robustness as a min–max problem and proposed Projected Gradient Descent (PGD): iterate small FGSM steps from a random start and project back onto the ϵ-ball after each step. PGD with a cross-entropy loss became the standard first-order attack. Carlini and Wagner [6] proposed margin-based losses that are more effective against models that saturate the cross-entropy, and showed that several defences that resisted earlier attacks were broken by a well-designed optimisation.

2.3.2 Adversarial training

The most reliable empirical defence is adversarial training: minimise the loss on adversarial examples generated on the fly [42]. Multi-step PGD training is expensive,

which motivated single-step alternatives. Tramèr et al. [64] proposed R+FGSM, which takes a small random step before the FGSM step to escape the sharp curvature near data points. Wong, Rice and Kolter [68] showed that FGSM with random initialisation, combined with a suitable step size, can match PGD training at a fraction of the cost ("fast is better than free"). This thesis uses this RFGSM recipe for all adversarially trained models (Section 3.4).

Adversarial training has well-documented side effects. Tsipras et al. [65] argue that robustness and standard accuracy can be fundamentally at odds, TRADES [71] makes the trade-off explicit with a regularised objective, and Rice, Wong and Kolter [51] show that robust accuracy can peak early and then decline during training (robust overfitting). We observe related effects in our SNNs: adversarially trained models lose 14–26 points of clean accuracy relative to clean-trained ones (Table 4.2), and adversarial training is unstable for the smallest backbone, where validation accuracy peaks well before the end of training and one seed's training accuracy collapses (Figure B.1).

2.3.3 Pitfalls of robustness evaluation

A recurring finding is that defences which appear robust are often only hard to attack with the particular attack used. Athalye, Carlini and Wagner [2] showed that many published defences relied on *obfuscated gradients* (shattered, stochastic or vanishing gradients) and could be broken by techniques such as backward-pass differentiable approximation and Expectation over Transformation (EoT) [3]. Carlini et al. [5] distilled the lessons into a checklist for evaluating robustness, and Tramèr et al. [63] showed in detail how thirteen defences could be circumvented by attacks adapted to their specific mechanism.

To remove the dependence on attack hyperparameters, Croce and Hein [10] proposed AutoAttack, an ensemble of four complementary, parameter-free attacks. Two are versions of APGD, one with a cross-entropy loss and one with a targeted difference-of-logits-ratio loss, and both adapt their step size automatically. The third is the targeted Fast Adaptive Boundary attack [9], which searches for minimal perturbations. The fourth is the query-based black-box Square Attack [1], which does not use gradients at all and therefore cannot be fooled by gradient masking. AutoAttack routinely lowers reported robust accuracy by several points relative to PGD, and it is the standard of RobustBench [8]. The gap between PGD and AutoAttack is a central quantity in this thesis: it turns out to hide VT's robustness cost entirely (Section 4.7).

2.4 Adversarial Robustness of Spiking Neural Networks

2.4.1 Early claims of inherent robustness

Sharmin et al. [56] provided one of the first comprehensive analyses of SNN robustness, finding that SNNs trained with spike-based methods could be more robust than equivalent ANNs to gradient-based attacks, especially in the black-box setting. Sharmin et al. [57] attributed this inherent robustness to two properties: discrete input encoding (Poisson rate coding introduces randomness) and the non-linear, leaky dynamics of spiking neurons. HIRE-SNN [34] built on this view by training SNNs with crafted input noise, improving robustness without the full cost of adversarial training.

2.4.2 Robust training and certification

Liang et al. [38] adapted bound-propagation-based certified training to SNNs, providing verifiable robustness guarantees for spiking models. RSC-SNN [69] observed that Poisson rate coding behaves like randomised smoothing, and proposed randomised smoothing coding to improve the robustness–accuracy trade-off. RandHet-SNN [67] introduced randomly heterogeneous spiking neurons as an adversarial defence. The experimental code base used in this thesis descends from its BrainCog implementation, although we deliberately disable all stochastic components (Section 3.12). Ding et al. [15] argue more broadly that the computational paradigms of neuromorphic systems can contribute to robustness.

2.4.3 Reliable evaluation of SNN robustness

The same evaluation pitfalls that affected ANN defences also apply to SNNs, and more severely. Surrogate gradients are an approximation chosen for training, and an attacker who uses the training surrogate may obtain weak or vanishing gradients. Stochastic encodings and neurons introduce exactly the kind of randomness that EoT was designed to handle. Wang et al. [66] show that standard gradient-based attacks can substantially over-estimate SNN robustness because of poorly matched surrogate gradients, and propose attacks that adapt the surrogate to obtain more reliable estimates. Their conclusion, that much reported SNN robustness reflects weak evaluation rather than genuine resistance, motivates our use of deterministic neurons (no EoT needed), AutoAttack (including the gradient-free Square attack), and an attacker that differentiates through the full trace of per-timestep outputs.

2.4.4 What is missing

All of the works above evaluate fixed-horizon SNNs. None studies an SNN whose horizon depends on the input, so none asks whether the stopping decision itself changes robustness, whether an adversary can exploit it, or how robustness relates to the number of timesteps actually run.

2.5 Attacks on the Computation of Adaptive Networks

Adaptive networks expose a second target besides the prediction: the amount of computation. Haque et al. [23] introduced ILFO, an attack on adaptive neural networks that perturbs inputs to increase the computation they trigger, by optimising intermediate outputs that control the network's gating decisions. Hong et al. [27] studied multi-exit networks specifically and showed that their DeepSloth attack could almost entirely eliminate the computational savings of early exit, in some cases while preserving the correct label, and that such slowdown perturbations transfer across models. Shumailov et al. [58] generalised the idea to *sponge examples*, inputs crafted to maximise the energy consumption and latency of a network, and demonstrated them on hardware accelerators.

These attacks were developed for ANNs whose adaptive behaviour comes from gating modules or side classifiers. VT in SNNs is a particularly exposed case: its exit decision is a simple threshold on an output-derived confidence score, the efficiency gain is the main reason to use it, and on neuromorphic hardware energy scales directly with the number of timesteps executed. Whether compute-overuse attacks transfer to VT SNNs, and whether adversarial training defends against them, has not been studied.

2.6 Variance and Reporting Practice

The final strand concerns how reliable reported improvements are. Henderson et al. [25] showed in deep reinforcement learning that random seeds alone can produce differences as large as those claimed between methods. Melis, Dyer and Blunsom [43] found that a well-tuned baseline could outperform more recent language models, and Dodge et al. [16] argued that results should be reported as a function of tuning budget. Lipton and Steinhardt [39] discussed broader troubling trends, including the failure to identify the sources of empirical gains. Bouthillier et al. [4] decomposed benchmark variance into its sources (data sampling, initialisation, data order, hyperparameter choice) and recommended randomising as many as possible and testing

paired differences. Picard [49] showed that on CIFAR-10, the choice of random seed alone produces a spread of accuracy large enough to make many published differences meaningless.

Two practices from this literature directly shape our methodology: comparing methods through *paired* differences on the same checkpoint, which cancels much of the seed variance, and replicating key claims across independently trained seeds rather than reporting a single run.

2.7 Synthesis and Research Gap

Table 2.1 summarises the closest prior work along the dimensions that matter for this thesis. Three gaps stand out.

1. **No compute-matched control.** VT methods are compared with fixed-horizon SNNs at the full horizon, or at a few horizons plotted on the same axes, but not systematically with the fixed horizon that uses the same measured compute. Without this control, the efficiency gain of choosing *when* to stop cannot be separated from the gain of simply stopping earlier.

2. **No adversarial evaluation of the exit gate.** SNN robustness studies evaluate fixed-horizon models, while VT studies evaluate clean data. How VT changes robustness, and whether its gate can be attacked, is unexplored, although analogous attacks on multi-exit ANNs are well documented.

3. **Single-run, single-selection conclusions.** Reported VT gains are typically single runs at a single operating point. How the conclusions depend on the selection of that point, on the strength of the attack, and on training-seed variance is not examined.

Positioning. This thesis does not propose a new VT method. It takes the simplest and most common form of VT, a maximum-softmax threshold on accumulated logits, and studies it under a protocol designed to answer the three gaps above: every VT point is compared with its compute-matched fixed horizon, robustness is measured with AutoAttack and with gate-specific attacks by an attacker that sees the full trace, and the key comparisons are replicated across training seeds and tested for sensitivity to the selection rule. The closest related work is DeepSloth [27], which established compute-inflation attacks on multi-exit ANNs. We transfer that threat model to VT in SNNs and relate it to training regime. Our conclusions about efficiency are there-

Table 2.1: Positioning of this thesis against the closest prior work.

Work	SNN	Adaptive T	Compute-matched	Strong attack	Gate attack	Multi-seed
Dynamic Confidence [36]	✓	✓	∘	—	—	—
DT-SNN [37]	✓	✓	∘	—	—	—
SpikeCP [7]	✓	✓	∘	—	—	—
HIRE-SNN [34]	✓	—	—	∘	—	—
RSC-SNN [69]	✓	—	—	✓	—	—
Reliable SNN eval. [66]	✓	—	—	✓	—	—
DeepSloth [27]	—	✓	—	✓	✓	—
ILFO [23]	—	✓	—	—	✓	—
This thesis	✓	✓	✓	✓	✓	✓

✓ addressed ∘ partially addressed — not addressed

fore complementary to, not in competition with, the VT methods reviewed above: we find that their central premise holds under a stricter comparison, and identify the robustness cost and attack surface that accompany it.

2.8 Summary

The literature establishes that VT can make SNNs cheaper, that SNN robustness must be measured with strong and adapted attacks, and that adaptive computation can itself be attacked. It does not establish whether VT is better than a fixed horizon at equal compute, how VT interacts with adversarial robustness, or whether these conclusions are robust to selection and seed variance. The next chapter describes the methodology we use to answer these questions.

Chapter 3

Methodology

This chapter describes how the study was carried out. Section 3.1 gives an overview of the pipeline. Section 3.2 formalises spiking inference, fixed-timestep inference and variable timing. Section 3.3 describes the three backbones and Section 3.4 the training regimes and schedules. Section 3.5 describes the data splits. Section 3.6 defines the compute metric, Section 3.7 the compute-matched evaluation protocol and Section 3.8 the operating-point selection rules. Section 3.9 describes the stopping-rule comparison, Section 3.10 the threat model and attacks, and Section 3.11 the multi-seed design and statistical tests. Section 3.12 documents the implementation.

3.1 Overview

Figure 3.1 summarises the pipeline. Each model configuration is defined by a backbone, a training regime and a timestep schedule. After training, the model is evaluated in two inference modes on a held-out calibration split: fixed-timestep inference at every horizon $T \in \{1, \dots, 8\}$, and VT inference at every threshold θ in a fixed grid. Each VT point is paired with the fixed horizon of nearest measured compute. An operating point is selected on the calibration split and then reported on the test split, where it is also attacked. The adversarially trained configurations are additionally re-trained with three seeds to test which conclusions replicate.

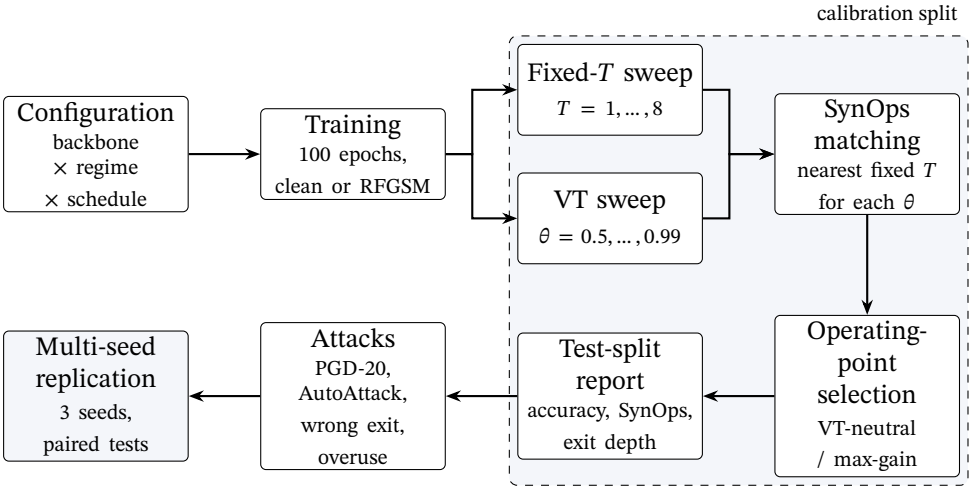


Figure 3.1: Overview of the experimental pipeline. Sweeps and operating-point selection use the calibration split, and all reported numbers use the test split.

3.2 Problem Formulation

3.2.1 Spiking neurons

All networks in this thesis use deterministic leaky integrate-and-fire neurons. Let I_t be the input current to a neuron at timestep t (the output of the preceding convolution and batch normalisation), V_t its membrane potential and $S_t \in \{0, 1\}$ its output spike. With time constant τ and firing threshold V_{th}, the neuron is updated as

$$H_t = V_{t-1} + \frac{1}{\tau}(I_t - V_{t-1}), \tag{3.1}$$

$$S_t = \Theta(H_t - V_{\text{th}}), \tag{3.2}$$

$$V_t = H_t(1 - S_t), \tag{3.3}$$

where Θ is the Heaviside step function and $V_0 = 0$. Equation (3.1) integrates the input with leak, Equation (3.2) emits a spike when the charged potential reaches the threshold, and Equation (3.3) resets the potential to zero after a spike (hard reset). We use $\tau = 2$ and $V_{\text{th}} = 0.5$ throughout.

The derivative of Θ is replaced during backpropagation by the surrogate

$$\frac{\partial S_t}{\partial H_t} \approx \frac{\alpha}{2(1 + \alpha |H_t - V_{\text{th}}|)^2}, \qquad \alpha = 4, \tag{3.4}$$

and the reset term is detached from the computational graph, a common choice that improves training stability. Because the neurons are deterministic, the same input always produces the same spike train, so there is no stochastic component for an at-

tacker to average over.

3.2.2 Input encoding and readout

We use direct encoding: the normalised image x is fed to the first convolutional layer unchanged at every timestep, and the first layer of LIF neurons converts it into spikes. All subsequent layers communicate by spikes, except the final linear classifier, which produces real-valued logits $z_t \in \mathbb{R}^K$ at every timestep, with $K = 10$ classes.

The network's prediction after t timesteps is based on the running mean of these logits,

$$\bar{z}_t = \frac{1}{t}\sum_{s=1}^{t} z_s, \qquad \hat{y}_t = \arg\max_k \ \bar{z}_{t,k}. \tag{3.5}$$

Because the membrane potentials start at zero and need time to charge, the deep layers of the network emit few or no spikes at $t = 1$, and $\bar{z}_1$ is nearly independent of the input. As a result, fixed-horizon accuracy at $T = 1$ is at or near chance for most of our models (Table 4.2).

3.2.3 Fixed-timestep inference

Fixed-timestep inference with horizon T runs every input for exactly T timesteps and predicts $\hat{y}_T$. Its compute is the same for every input (up to differences in spike sparsity) and grows roughly linearly with T. Sweeping T from 1 to 8 traces the fixed-timestep compute–accuracy curve, which serves as the baseline for VT.

3.2.4 Variable timing

Variable timing adds a stopping rule. Let $c(\cdot)$ map accumulated logits to a confidence score in $[0, 1]$, and let θ be a threshold. The exit time of input x is

$$\tau_\theta(x) = \min\{t \in \{1, \dots, T_{\max}\} \ : \ c\,(\bar{z}_t(x)) \geq \theta\}, \tag{3.6}$$

with $\tau_\theta(x) = T_{\max}$ if no such t exists, and the VT prediction is $\hat{y}_{\tau_\theta(x)}$. We use $T_{\max} = 8$. The main stopping rule is the maximum softmax probability,

$$c_{\mathrm{MSP}}(\bar{z}) = \max_k \mathrm{softmax}(\bar{z})_k, \tag{3.7}$$

which is the most common choice in the VT literature and a strong baseline in confidence estimation generally [26]. Alternative rules are compared in Section 3.9.

Figure 3.2 illustrates the mechanism. After every timestep, the confidence of the accumulated prediction is compared with θ. An easy input crosses the threshold early and exits, while a hard input continues until it crosses the threshold or reaches $T_{\max}$. The average exit depth $\mathbb{E}_x[\tau_\theta(x)]$ and the average compute fall as θ decreases.

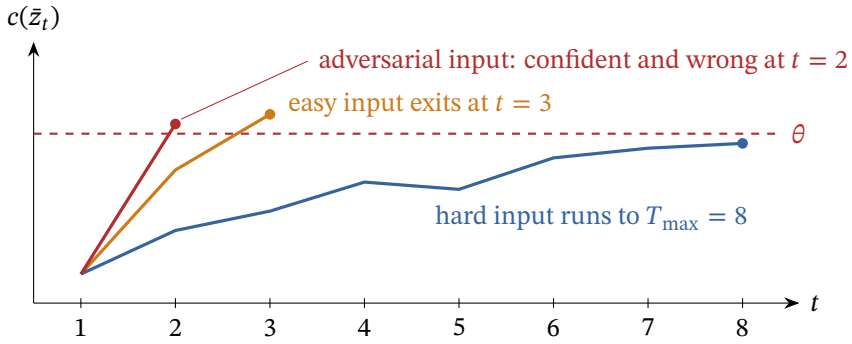


Figure 3.2: Variable-timing inference. The confidence of the accumulated prediction is checked after every timestep, and an input exits as soon as it reaches the threshold θ. An adversary can try to make an input exit early with a wrong prediction (red), or to keep its confidence below θ so that it never exits early (Section 3.10).

Sample-exact execution. In a batched implementation, a batch keeps running until all its inputs have exited, and exited inputs simply carry their frozen predictions. This does not change predictions, but it does change measured compute: a batch-level measurement charges every input for the slowest input's timesteps. We therefore measure compute with *sample-exact* accounting, in which each input's synaptic operations are counted only up to its own exit time (Section 3.6). The reported VT compute is exactly what a per-sample implementation on event-driven hardware would incur.

3.3 Backbone Architectures

We use spiking versions of three standard convolutional architectures, chosen to span a range of depth, width and parameter count. In every case, each ReLU of the original architecture is replaced by an LIF neuron (Equation (3.1)–(3.3)), and the final classifier is a real-valued linear layer. Table 3.1 summarises them.

- **ResNet-18** [24], CIFAR variant: a 3×3 stem convolution with 64 channels followed by four stages of two basic residual blocks each with 64, 128, 256 and 512 channels, global average pooling and a linear classifier. LIF neurons follow every batch-normalised convolution, including on the shortcut path.
- **VGG16** [59]: thirteen 3×3 convolutions in five stages (2, 2, 3, 3 and 3 layers with 64, 128, 256, 512 and 512 channels), each followed by batch normalisation [30]

and an LIF neuron, with max pooling between stages and a classifier with two spiking hidden layers of 4096 units.

- **MobileNetV2** [54]: inverted residual blocks with depthwise-separable convolutions in seven stages (16, 24, 32, 64, 96, 160 and 320 output channels with 1, 2, 3, 4, 3, 3 and 1 blocks), with LIF neurons replacing the ReLU6 activations.

Table 3.1: Backbones used in this thesis. Dense operations are the multiply–accumulate operations a non-spiking network of the same shape would perform over eight timesteps, per image.

Backbone	Parameters	Dense ops at $T = 8$	Full-horizon SynOps (clean model)
ResNet-18	11.2 M	4.44×10^9	4.14×10^8
VGG16	33.6 M	2.66×10^9	1.82×10^8
MobileNetV2	2.2 M	0.70×10^9	3.77×10^8

The SynOps column of Table 3.1 already shows the effect of spike sparsity: a spiking ResNet-18 at its full horizon performs fewer than a tenth of the operations of its dense counterpart, because most activations are zero at any given timestep. MobileNetV2, despite having the fewest dense operations, performs more synaptic operations than VGG16, because its depthwise and expansion layers are much less sparse.

3.4 Training

3.4.1 Training regimes

Every backbone is trained under two regimes.

Clean. The model is trained on clean images with the cross-entropy loss $\mathcal{L}_{\mathrm{CE}}(f_T(x), y)$ of the accumulated logits after the training horizon T.

Adversarial. The model is trained entirely on adversarial examples generated on the fly with FGSM from a random start (RFGSM) [64], [68]. For each training image, a random perturbation $\delta_0 \sim \mathcal{U}[-\gamma\epsilon, \gamma\epsilon]^d$ is drawn, a single signed gradient step is taken, and the result is projected onto the ϵ-ball:

$$x_{\mathrm{adv}} = \mathrm{clip}_{[0,1]}(x + \Pi_\epsilon[\delta_0 + \alpha\,\mathrm{sign}(\nabla_\delta \mathcal{L}_{\mathrm{CE}}(f_T(x + \delta_0), y))]), \tag{3.8}$$

with $\epsilon = \alpha = 8/255$ and $\gamma = 0.1$. The model is then updated on $\mathcal{L}_{\mathrm{CE}}(f_T(x_{\mathrm{adv}}), y)$ only, and no clean images are used.

The two regimes are deliberately pure: every checkpoint in this thesis was trained with a fixed clean fraction of exactly 0% or 100%, so that any difference between them can be attributed to the regime itself.

3.4.2 Timestep schedules

The training horizon is set by one of two schedules.

Fixed-T8. Every training batch uses $T = 8$. This is the standard setting for directly trained SNNs and the schedule used for all three backbones.

Mixed-T. For every training batch, T is drawn uniformly from $\{1, \dots, 8\}$. Training at short horizons should make early predictions more useful, which matters for VT because VT relies on them. We train mixed-T models for ResNet-18 under both regimes to test this.

3.4.3 Optimisation and hyperparameters

All models are trained for 100 epochs with stochastic gradient descent (momentum 0.9, weight decay 10^{-4}, base learning rate 0.1, batch size 256), with a linear warm-up over the first five epochs followed by cosine decay [40]. We use automatic mixed precision with float16 [45] and clip the gradient norm at 1.0. Training images are augmented with random cropping (4-pixel padding), horizontal flipping and Cutout [14] with one 16×16 hole. Clean validation accuracy at the full horizon is evaluated every five epochs, and the checkpoint with the highest validation accuracy is kept. Table A.1 in Appendix A lists every hyperparameter.

3.4.4 Model configurations

Table 3.2 lists the nine trained configurations. ResNet-18 is the primary backbone: it is trained under both regimes and both schedules, and the adversarial fixed-T8 configuration was trained twice with identical settings. VGG16 and MobileNetV2 are trained with the fixed-T8 schedule under both regimes.

3.5 Data and Splits

All experiments use CIFAR-10 [33]: 50,000 training and 10,000 test images of size 32×32 in ten classes. We split the official training set with a fixed seed into three

Table 3.2: Model configurations. The short names are used in the results tables and figures.

Short name	Backbone	Regime	Schedule	Note
R18 clean	ResNet-18	Clean	Fixed-T8	
R18 adv (1)	ResNet-18	Adversarial	Fixed-T8	first run
R18 adv (2)	ResNet-18	Adversarial	Fixed-T8	identical re-run
R18 clean mixT	ResNet-18	Clean	Mixed-T	
R18 adv mixT	ResNet-18	Adversarial	Mixed-T	
VGG clean	VGG16	Clean	Fixed-T8	
VGG adv	VGG16	Adversarial	Fixed-T8	
MBv2 clean	MobileNetV2	Clean	Fixed-T8	
MBv2 adv	MobileNetV2	Adversarial	Fixed-T8	

disjoint parts (Figure 3.3): 40,000 images for training, 5,000 for validation (checkpoint selection) and 5,000 for calibration (threshold sweeps and operating-point selection). The 10,000 test images are used only for reporting. Separating calibration from test ensures that the operating point is chosen without looking at the data on which it is reported.

Adversarial evaluation is expensive, so attacks are run on a class-balanced subset of the test set: 2,000 images (200 per class) for the single-run experiments and 1,000 images (100 per class) for the multi-seed experiments, drawn with a fixed seed. The earliest ResNet-18 experiments were attacked on the full test set, which is noted where relevant.

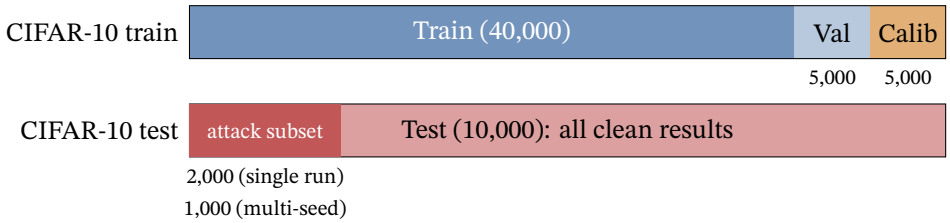


Figure 3.3: Data splits. Training uses 40,000 images. Checkpoints are selected on validation, operating points on calibration, and all results are reported on test. Attacks use a class-balanced test subset.

3.6 Compute Accounting

We measure compute in synaptic operations. For a convolutional or linear layer ℓ executed at timestep t, let D_ℓ be the number of multiply–accumulate operations a dense execution would perform and $\rho_{\ell,t}(x)$ the fraction of the layer's input activations that are non-zero. The synaptic operations of the layer are $D_\ell\,\rho_{\ell,t}(x)$: only non-zero

inputs trigger work. For an input that runs for $\tau(x)$ timesteps,

$$\mathrm{SynOps}(x) = \sum_{t=1}^{\tau(x)} \sum_{\ell} D_\ell \, \rho_{\ell,t}(x), \tag{3.9}$$

and we report the mean over the evaluation set. For the first layer, whose input is the real-valued image, $\rho = 1$, so its dense cost is counted in full at every timestep. For fixed-timestep inference $\tau(x) = T$, and for VT $\tau(x) = \tau_\theta(x)$, measured per sample. SynOps is a hardware-independent proxy: it captures the effect of both sparsity and exit depth, but not memory access or membrane-update costs [11]. We use it to compare VT and fixed-timestep inference on the same network, where those omitted costs scale similarly with the number of timesteps run.

3.7 Compute-Matched Evaluation Protocol

For each trained model, we run two sweeps on the calibration split and repeat them on the test split:

- a *fixed sweep* at $T \in \{1, \dots, 8\}$, recording accuracy and mean SynOps, and
- a *VT sweep* at $\theta \in \Theta = \{0.5, 0.6, 0.7, 0.8, 0.85, 0.9, 0.95, 0.99\}$, recording accuracy, mean exit depth, the histogram of exit times and mean SynOps.

Each VT point θ is then paired with its *compute-matched control*: the fixed horizon whose mean SynOps is closest to that of VT at θ,

$$T^*(\theta) = \arg\min_{T \in \{1,\dots,8\}} \left| \mathrm{SynOps}_{\mathrm{VT}}(\theta) - \mathrm{SynOps}_{\mathrm{fix}}(T) \right|. \tag{3.10}$$

The *gain* of VT at θ is its accuracy minus that of the matched control,

$$\Delta(\theta) = \mathrm{Acc}_{\mathrm{VT}}(\theta) - \mathrm{Acc}_{\mathrm{fix}}\left(T^*(\theta)\right). \tag{3.11}$$

A positive gain means that, for roughly the same compute, choosing when to stop per input is better than stopping every input at the same time. Because the fixed horizons are discrete, the match is approximate. We report both SynOps values so that the residual mismatch is visible, and we additionally report whether each VT point lies above the piecewise-linear fixed-timestep frontier.

We also report VT against the *full-horizon* model ($T = 8$). This comparison answers a different question, namely what VT gives up relative to running the whole budget,

and both are needed to interpret the robustness results.

3.8 Operating-Point Selection

Every reported VT result is at some threshold, and the threshold must be selected without looking at test data. We consider two selection rules, both applied to the calibration sweeps.

Max-gain selection. The rule used in our original experiments selects the threshold with the largest gain over its matched control on the calibration split, $\hat{\theta}_{\text{gain}} = \arg\max_{\theta} \Delta_{\text{cal}}(\theta)$. This rule is natural, since it picks the point where VT looks best, but it is *biased* in favour of VT: it rewards VT on exactly the comparison it is being evaluated on, and it can prefer points where both VT and its control are poor, as long as the control is poorer. We show in Section 4.6 that this happens in practice.

VT-neutral selection. The alternative rule ignores the matched control entirely. Given a tolerance δ, it selects the cheapest threshold whose calibration accuracy is within δ points of the full-horizon accuracy:

$$\hat{\theta}_{\delta} = \arg\min_{\theta \in \Theta} \left\{ \text{SynOps}_{\text{VT,cal}}(\theta) \;:\; \text{Acc}_{\text{VT,cal}}(\theta) \geq \text{Acc}_{\text{fix,cal}}(8) - \delta \right\}. \tag{3.12}$$

If no threshold meets the tolerance, the most accurate threshold is chosen. This is the rule a practitioner would actually use: decide how much accuracy one is willing to give up, then take the cheapest setting that meets it. Because it never looks at the fixed-timestep controls, it cannot favour VT in the comparison with them. Algorithm 1 gives the full procedure. We use $\delta = 1$ point as the primary setting and report $\delta \in \{0.5, 2\}$ as a sensitivity analysis.

Which rule was used where. The clean-data results in this thesis are recomputed from the saved calibration and test sweeps under both rules, which requires no new model evaluations. The attack experiments, however, were run at the thresholds chosen by the max-gain rule (with one manual override, noted in Section 4.6), together with their immediate neighbours in Θ. Re-running them at the VT-neutral thresholds was not possible within our compute budget. For the adversarially trained models, the two rules select the same or adjacent thresholds (Table 4.5), so this affects the clean-trained robustness results most, and those are near zero under every rule.

Algorithm 1 VT-neutral operating-point selection and compute-matched reporting

Require: trained model f; threshold grid Θ; tolerance δ; calibration set $\mathcal{D}_c$; test set $\mathcal{D}_t$

for $T = 1, \dots, 8$ **do** ▷ fixed sweep
  $(a_T^c, s_T^c) \leftarrow$ accuracy and mean SynOps of f at horizon T on $\mathcal{D}_c$
end for
for $\theta \in \Theta$ **do** ▷ VT sweep, sample-exact
  $(v_\theta^c, u_\theta^c) \leftarrow$ accuracy and mean SynOps of $\mathrm{VT}(f, \theta)$ on $\mathcal{D}_c$
end for
$\mathcal{C} \leftarrow \{\theta \in \Theta : v_\theta^c \geq a_8^c - \delta\}$
if $\mathcal{C} = \emptyset$ **then**
  $\hat{\theta} \leftarrow \arg\max_\theta v_\theta^c$ ▷ fallback
else
  $\hat{\theta} \leftarrow \arg\min_{\theta \in \mathcal{C}} u_\theta^c$
end if
Evaluate $\mathrm{VT}(f, \hat{\theta})$ and fixed $T = 1..8$ on $\mathcal{D}_t$
$T^* \leftarrow \arg\min_T |u_{\hat{\theta}}^t - s_T^t|$ ▷ compute-matched control
return $v_{\hat{\theta}}^t - a_{T^*}^t, \;\; v_{\hat{\theta}}^t - a_8^t, \;\; u_{\hat{\theta}}^t / s_8^t$

3.9 Stopping Rules

To test whether the conclusions depend on the choice of confidence score, we compare five stopping rules, each computed on the accumulated logits $\bar{z}_t$ with $p = \mathrm{softmax}(\bar{z}_t)$:

- **Max softmax**: $c = \max_k p_k$ (the primary rule, Equation (3.7)).
- **Margin**: $c = p_{(1)} - p_{(2)}$, the difference between the two largest probabilities.
- **Entropy**: $c = 1 - H(p)/\log K$, one minus the normalised predictive entropy.
- **Temperature softmax**: $c = \max_k \mathrm{softmax}(\bar{z}_t/2)_k$, MSP with temperature 2.
- **Evidential surrogate**: with $\alpha_k = \mathrm{softplus}(\bar{z}_{t,k}) + 1$, $c = 1 - K/\sum_k \alpha_k$, one minus the Dirichlet uncertainty mass [55].

Because the rules have different scales, comparing them at the same θ is meaningless. Instead, for each rule we choose the threshold whose mean exit depth on the calibration split is closest to that of the max-softmax operating point, and compare test accuracy at (approximately) equal depth. A rule that is better at identifying inputs that are already correctly classified will achieve higher accuracy at the same depth.

3.10 Threat Model and Attacks

3.10.1 Threat model

We consider a white-box adversary who knows the model weights, the stopping rule and the threshold, and who perturbs each test image within an ℓ_∞ ball of radius $\epsilon = 8/255$, keeping pixel values in $[0, 1]$. The adversary has one of three goals:

1. **Misclassification**: make the final prediction wrong (untargeted attack).
2. **Wrong early exit**: make the input exit early with a wrong prediction.
3. **Compute overuse**: keep the prediction *correct* but delay the exit, increasing the compute spent on the input.

The first goal is the standard robustness setting. The second and third are specific to VT and target its exit gate.

3.10.2 Attacking through the exit gate

The exit decision in Equation (3.6) is a hard threshold, so it has no gradient. An attacker who differentiates only through the full-horizon output $\bar{z}_8$ is attacking a different function from the one being deployed. We therefore give the attacker the full trace: for each candidate perturbation, the network is run for all eight timesteps, the exit time $\tau_\theta(x)$ is determined from the per-timestep confidences, and the loss is computed on the accumulated logits $\bar{z}_{\tau_\theta(x)}$ at the actual exit step. Gradients flow through those logits and through all earlier timesteps' spiking dynamics, but not through the choice of exit step itself. This is an adaptive attack in the sense of Tramèr et al. [63]: it attacks the deployed decision rule, not a proxy. Because our neurons are deterministic, no Expectation over Transformation is needed.

3.10.3 Misclassification attacks

PGD-20. Projected gradient descent [42] with 20 iterations, step size $2/255$, a uniform random start in the ϵ-ball and the cross-entropy loss on the exit-step logits.

AutoAttack. The standard version of AutoAttack [10]: APGD-CE, APGD-T, FAB-T [9] and Square [1], applied in sequence, each to the images that the previous attacks failed to break. The model is wrapped so that AutoAttack sees the VT decision rule through the exit-step logits as above.

Each attack is applied to three inference modes on the same images: the full-horizon model ($T = 8$), the compute-matched fixed horizon T^*, and VT at the selected threshold. The fixed-mode models are attacked through their own T-step outputs.

3.10.4 Gate-specific attacks

Both gate-specific attacks use PGD with the same budget as PGD-20 but a different loss computed over the whole trace. Let $m_t(x) = \max_{k \neq y} \bar{z}_{t,k} - \bar{z}_{t,y}$ be the misclassification margin of the accumulated logits at step t (positive when the prediction is wrong) and $c_t = c(\bar{z}_t)$ the confidence.

Wrong early exit. The attacker maximises a weighted sum of margin and confidence that emphasises early timesteps,

$$\mathcal{L}_{\text{early}} = \frac{1}{T_{\max}} \sum_{t=1}^{T_{\max}} w_t \, (m_t + \lambda_{\text{c}} \, c_t), \qquad w_t = \frac{T_{\max} - t + 1}{T_{\max}}, \tag{3.13}$$

with $\lambda_{\text{c}} = 0.25$, encouraging predictions that are wrong and confident as early as possible.

Compute overuse. The attacker maximises the normalised entropy $\tilde{H}_t$ and minimises the confidence of the accumulated prediction at every timestep, while preserving the correct final label:

$$\mathcal{L}_{\text{overuse}} = \frac{1}{T_{\max}} \sum_{t=1}^{T_{\max}} (\tilde{H}_t - c_t) \; - \; \lambda_{\text{p}} \, \mathcal{L}_{\text{CE}} \, (\bar{z}_\tau, y), \tag{3.14}$$

with $\lambda_{\text{p}} = 1$. The first term pushes confidence below the threshold so that the input does not exit, and the second keeps the prediction correct, so that the attack is invisible to anyone monitoring accuracy. We report the increase in mean exit depth (depth inflation) and the fraction of attacked inputs that are still correctly classified. This attack is the SNN analogue of slowdown attacks on multi-exit networks [27] and of sponge examples [58].

3.10.5 Attack variance

For one checkpoint per adversarially trained backbone, PGD-20 and AutoAttack were repeated with three attack seeds (different random starts, and different random choices inside AutoAttack). The standard deviation across attack seeds was at most 1.74 points

for every mode and backbone (Table B.14), much smaller than the training-seed variance, so the remaining experiments use a single attack seed.

3.11 Multi-Seed Replication and Statistical Analysis

3.11.1 Multi-seed design

Single-run results can be dominated by training variance. To test which conclusions replicate, the adversarially trained fixed-T8 configurations were re-trained from scratch with three seeds ($s \in \{0, 1, 2\}$) for ResNet-18 and VGG16, using the same pure-adversarial regime as the single-run models. For each seed, the operating point and its compute-matched control were selected on the calibration split, and clean accuracy, PGD-20 and AutoAttack were measured for the full-horizon, matched and VT modes on the same 1,000-image balanced test subset. A MobileNetV2 multi-seed run was also started but exceeded the 12-hour Kaggle session limit before its evaluations completed, so only its training histories and attack-seed variance are available.

The multi-seed notebooks select operating points with the max-gain rule and save only the selected point, not the full sweeps, so they cannot be re-selected with the VT-neutral rule. The selected points are nevertheless sensible: for every seed, the VT point is within about three points of the full-horizon accuracy on clean data, and the matched horizons ($T^* = 5$ for ResNet-18, $T^* = 6$ for VGG16) are the same as those of the single-run VT-neutral points.

3.11.2 Paired differences

All comparisons are *paired*: VT, its matched control and the full-horizon model are evaluated on the same checkpoint and the same images, and we analyse the per-checkpoint difference $d_s = \mathrm{Acc}_{\mathrm{VT},s} - \mathrm{Acc}_{\mathrm{ref},s}$. Pairing removes the variation between checkpoints, which is large (up to 7 points in full-horizon AutoAttack accuracy across our ResNet-18 seeds), and isolates the effect of the inference mode [4].

3.11.3 Tests

For each comparison and metric we report the mean and sample standard deviation of d_s over checkpoints, the number of checkpoints with $d_s > 0$, and two tests of the null hypothesis that the mean difference is zero:

- a one-sample t-test on $\{d_s\}$, with $t = \bar{d}/(s_d/\sqrt{n})$ and $n-1$ degrees of freedom, and
- an exact two-sided sign test (binomial test on the number of positive differences), which makes no distributional assumption.

We pool the three ResNet-18 and three VGG16 seeds ($n = 6$) for the main tests and also report each backbone separately. With so few checkpoints these tests have limited power: a non-significant result means the data do not distinguish the difference from zero, not that the difference is zero. We therefore interpret them together with the sign of every individual seed.

For single-run accuracies on n images, we report the binomial 95% confidence half-width $1.96\sqrt{p(1-p)/n}$ as a guide to evaluation noise: about $\pm$0.6–1.0 points for clean accuracy on the 10,000-image test set and about $\pm$2 points for attacked accuracy on a 2,000-image subset. This captures only test-sampling noise. Training-seed variance, which is larger, is addressed by the multi-seed experiments.

3.12 Implementation and Reproducibility

Code base. The experimental suite is written in PyTorch [48]. It descends from the BrainCog implementation of RandHet-SNN [67], from which we retain the data pipeline and training scaffolding. We replaced the stochastic heterogeneous neurons with deterministic LIF neurons, removed the noise-scaling, Parseval and weight-perturbation components, and added the VT inference path with per-step traces, sample-exact compute accounting, the stopping rules, the compute-matched evaluation protocol, the gate-specific attacks and the multi-seed analysis. AutoAttack is run through the torchattacks library [32], with the reference AutoAttack implementation as a fallback.

Execution environment. All training and evaluation ran on Kaggle GPU sessions with internet access disabled. CIFAR-10 and the attack libraries were attached as offline datasets. Each experiment is a self-contained notebook generated by a builder script, so that the exact code that produced a result is preserved alongside it. Mixed precision, TF32 matrix multiplication and cuDNN autotuning were enabled for speed.

Analysis. Every number in Chapter 4 is computed by an analysis script from the raw CSV files written by the notebooks, and the script also writes a facts sheet that

the text was checked against. Figures are generated from the same data. Appendix C lists the files and the commands to regenerate them.

3.13 Summary

We study variable timing in deterministic LIF SNNs on CIFAR-10 across three backbones, two training regimes and two timestep schedules. Every VT operating point is compared with the fixed horizon of nearest measured compute, using sample-exact SynOps, and selected on a calibration split with a rule that does not favour VT. Robustness is measured with PGD-20 and AutoAttack by an attacker who differentiates through the exit-step logits, and two further attacks target the exit gate directly. Key comparisons are replicated across three training seeds and analysed with paired tests. The next chapter presents the results.

Chapter 4

Results and Discussion

This chapter presents the experimental results. Section 4.1 maps the experiments to the research questions and Section 4.2 summarises the setup and metrics. Section 4.3 characterises fixed-timestep inference, the baseline against which everything else is measured. Section 4.4 answers RQ1 on clean data, Section 4.5 tests whether the answer depends on the stopping rule, and Section 4.6 shows how it depends on the operating-point selection rule (RQ4). Section 4.7 turns to robustness (RQ2), Section 4.8 to the gate-specific attacks (RQ3), and Section 4.9 to the multi-seed replication that decides which of these findings hold. Section 4.10 discusses the results together, and Section 4.11 examines threats to validity.

4.1 Mapping Objectives to Experiments

Table 4.1 lists the experiments, the research questions they address and where their results appear.

4.2 Setup and Metrics

The setup is described in full in Chapter 3, and here we summarise the points needed to read the results. All models are spiking LIF networks with $T_{\max} = 8$, trained for 100 epochs on 40,000 CIFAR-10 images. Operating points are selected on a 5,000-image calibration split and reported on the 10,000-image test set. Attacks are ℓ_∞ with $\epsilon = 8/255$, run on a 2,000-image class-balanced test subset (1,000 for the multi-seed study). We report:

- **Accuracy** (%) on clean or attacked inputs.

Table 4.1: Experiments, the research questions they address, and where their results are reported.

Experiment	RQ	Configurations	Section
Fixed-timestep sweeps $T = 1..8$	—	all 9	4.3
VT sweeps and compute-matched comparison	RQ1	all 9	4.4
Fixed-T8 vs mixed-T training schedule	RQ1	ResNet-18 (4)	4.4.4
Stopping-rule comparison at matched depth	RQ1, RQ4	7 of 9	4.5
Max-gain vs VT-neutral selection; tolerance δ	RQ4	all 9	4.6
PGD-20 and AutoAttack, single run	RQ2	all 9	4.7
Confidence trajectories under attack	RQ2, RQ3	ResNet-18 adv	4.7.3
Wrong-early-exit and compute-overuse attacks	RQ3	all 9	4.8
Three-seed replication with paired tests	RQ1, RQ2, RQ4	ResNet-18, VGG16 adv	4.9
Attack-seed variance	RQ4	3 adv backbones	4.9

- **Mean SynOps** per image (Equation (3.9)), and the **SynOps saving** of VT relative to the full horizon, $1 - \text{SynOps}_{\text{VT}}/\text{SynOps}_{\text{fix}}(8)$.
- **Mean exit depth**: the average number of timesteps executed.
- **Gain**: VT accuracy minus the accuracy of the compute-matched fixed horizon T^* (Equation (3.11)).
- **Below full**: full-horizon accuracy minus VT accuracy, i.e. what VT gives up relative to running all eight timesteps.

All differences are in percentage points ("points"). As a guide, the binomial 95% half-width is $\pm$0.6–1.0 points for clean accuracy on the full test set and about $\pm$2 points for attacked accuracy on a 2,000-image subset.

4.3 Fixed-Timestep Behaviour

Table 4.2 and Figure 4.1 show test accuracy as a function of the fixed horizon T for all nine configurations. Four observations shape the rest of the chapter.

The first timestep carries no information for fixed-T8 models. Every model trained only at $T = 8$ is at chance (10%) at $T = 1$. With $V_0 = 0$, a leak of $1/\tau = 1/2$ and a threshold of 0.5, deep-layer neurons cannot reach threshold in a single step,

Table 4.2: Clean test accuracy (%) of fixed-timestep inference at each horizon T (n = 10,000).

Configuration	$T = 1$	2	3	4	5	6	7	8
R18 clean	10.00	84.72	87.89	90.11	90.48	90.97	91.09	91.26
R18 adv (1)	12.11	57.51	66.79	69.35	71.46	72.98	73.23	73.67
R18 adv (2)	10.00	58.74	65.93	72.30	73.53	76.01	76.32	77.18
R18 clean mixT	41.39	81.48	86.75	88.78	89.80	90.26	90.61	90.57
R18 adv mixT	46.28	70.04	74.31	76.26	76.82	77.14	77.58	77.79
VGG clean	10.00	70.94	80.50	84.63	86.16	87.50	87.82	88.20
VGG adv	10.00	23.81	51.41	57.62	61.23	63.75	64.62	65.52
MBv2 clean	10.00	17.60	37.72	61.06	64.91	73.92	74.71	77.66
MBv2 adv	10.00	23.47	36.55	45.54	47.63	50.09	50.64	51.44

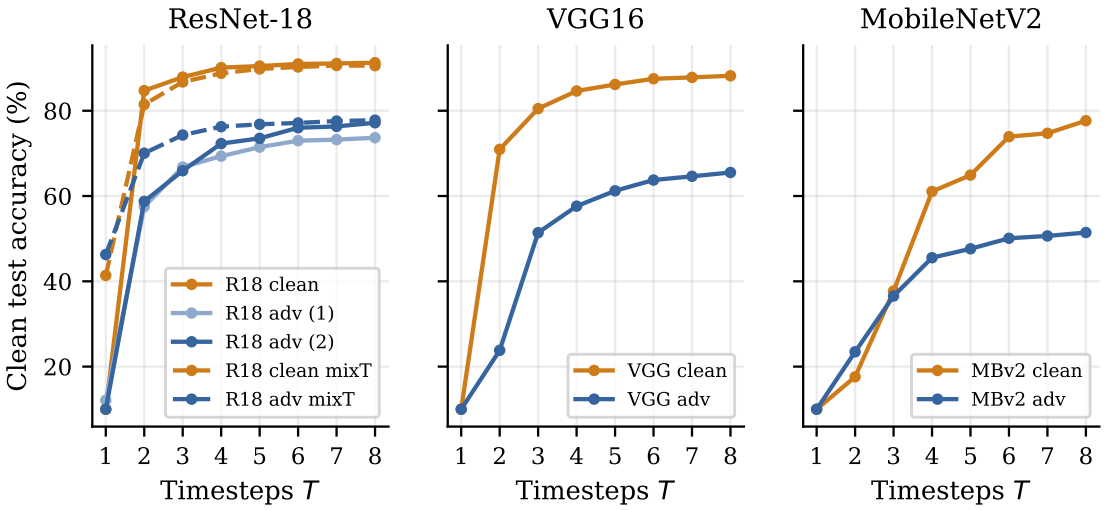


Figure 4.1: Clean test accuracy of fixed-timestep inference as a function of the horizon T. Solid lines show the fixed-T8 training schedule and dashed lines the mixed-T schedule.

so the output at $t = 1$ is essentially the classifier bias. Mixed-T models, which were trained to produce useful predictions at $T = 1$, reach 41–46% there.

Accuracy saturates, but at different rates. ResNet-18 clean reaches 87.89% at $T = 3$, within 3.4 points of its $T = 8$ accuracy. VGG16 clean needs $T = 5$ to come within about two points, and MobileNetV2 needs $T = 6$–8. Adversarially trained models saturate more slowly than their clean counterparts on every backbone. How quickly a backbone saturates bounds how much any early-exit method can save: if the fixed-timestep curve is flat after $T = 3$, there is little to gain by stopping at $T = 3$ for some inputs and $T = 8$ for others.

Adversarial training costs 14–26 points of clean accuracy. At $T = 8$, adversarial training reduces clean accuracy from 91.26% to 73.67–77.18% on ResNet-18, from 88.20% to 65.52% on VGG16 and from 77.66% to 51.44% on MobileNetV2, consistent with the robustness–accuracy trade-off reported for ANNs [65]. MobileNetV2 is the weakest backbone in both regimes, and its depthwise layers appear to suffer most from binarised activations.

Mixed-T training reshapes the curve. The effect of the mixed-T schedule depends on the regime. Under adversarial training it improves accuracy at every horizon, by up to 11 points at $T = 2$. Under clean training it lifts only $T = 1$ (41.39% vs 10.00%) and is slightly *worse* at every other horizon (for example 86.75% vs 87.89% at $T = 3$). Full-horizon accuracy is roughly unchanged in both regimes (90.57% vs 91.26% clean, 77.79% vs 77.18% adversarial). It also changes sparsity: the mixed-T clean ResNet-18 performs 5.65×10^8 SynOps at $T = 8$, 37% more than its fixed-T8 counterpart (4.14×10^8), because its neurons fire more. We return to the consequences for VT in Section 4.4.4.

4.4 RQ1: Efficiency at Equal Compute

4.4.1 The compute–accuracy frontier

Figure 4.2 shows the VT and fixed-timestep frontiers for the clean-trained ResNet-18, our primary configuration. Each orange point is one VT threshold and each blue point is one fixed horizon. At every compute level, the VT curve lies above the fixed-timestep curve. At the VT-neutral operating point ($\theta = 0.90$, $\delta = 1$), VT reaches 90.62% with a mean exit depth of 3.01 timesteps and 1.33×10^8 SynOps per image. The compute-matched fixed horizon, $T^* = 3$, uses 1.27×10^8 SynOps and reaches 87.89%. VT is therefore 2.73 points more accurate at slightly more than the same compute, and only 0.64 points below the full-horizon model while using 68% fewer SynOps.

Figure 4.3 repeats the comparison for all nine configurations, and Table 4.3 lists the VT-neutral operating points. The pattern is consistent: **at the VT-neutral operating point, VT is more accurate than its compute-matched fixed horizon on all nine configurations**, with gains from +0.14 to +3.37 points.

4.4.2 How large is the gain, and is it real?

The gains in Table 4.3 fall into three groups.

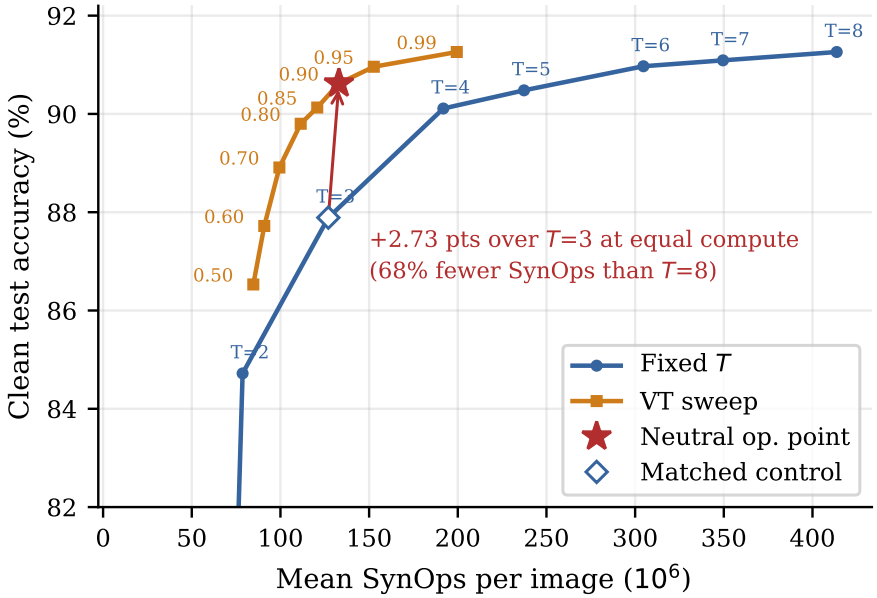


Figure 4.2: Compute–accuracy frontier for the clean-trained ResNet-18 (test set). Orange: VT at each threshold θ. Blue: fixed-timestep inference at each horizon. The star marks the VT-neutral operating point and the diamond its compute-matched fixed-timestep control.

Table 4.3: VT-neutral operating points ($\delta = 1$, selected on calibration, reported on test, $n = 10{,}000$). Depth: mean exit timestep. T^*: compute-matched fixed horizon. Gain: VT minus matched fixed accuracy. Below full: $T = 8$ accuracy minus VT accuracy. Saving: SynOps saved relative to $T = 8$. Above: number of VT thresholds (of 8) that lie above the fixed-timestep frontier. †No threshold met the tolerance, so the most accurate was used.

Configuration	θ	Depth	VT acc	T^*	Fixed acc	Gain	Below full	Saving	Above
R18 clean	0.90	3.01	90.62	3	87.89	+2.73	0.64	67.9%	8/8
R18 adv (1)	0.60	5.01	72.83	5	71.46	+1.37	0.84	42.0%	8/8
R18 adv (2)	0.60	5.13	76.42	5	73.53	+2.89	0.76	40.3%	8/8
R18 clean mixT†	0.99	4.24	88.92	4	88.78	+0.14	1.65	49.6%	3/8
R18 adv mixT	0.80	5.43	77.50	5	76.82	+0.68	0.29	35.6%	5/8
VGG clean	0.80	3.89	87.75	4	84.63	+3.12	0.45	59.3%	8/8
VGG adv	0.50	5.74	64.64	6	63.75	+0.89	0.88	35.0%	8/8
MBv2 clean	0.99	7.41	78.08	7	74.71	+3.37	−0.42	7.4%	4/8
MBv2 adv	0.60	7.21	50.92	7	50.64	+0.28	0.52	10.0%	7/8

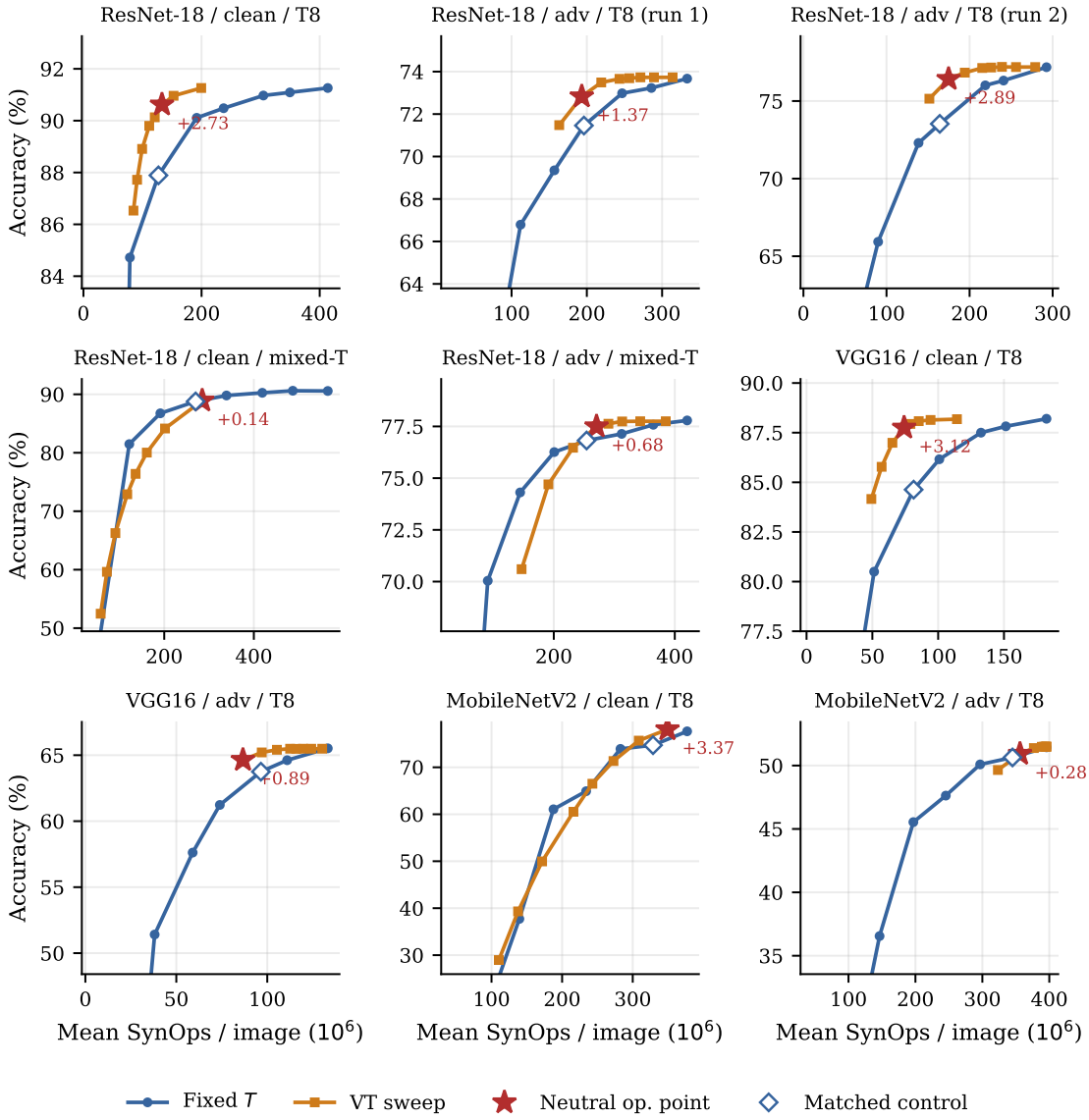


Figure 4.3: Compute–accuracy frontiers for all nine configurations (test set). Annotations give the gain of the VT-neutral operating point ($\delta = 1$) over its compute-matched control. The y-axes are cropped to the region of interest, so fixed-timestep points at $T = 1$ (and $T = 2$ where far below) lie off the bottom of several panels.

Clear gains with large savings. On the clean-trained fixed-T8 ResNet-18 and VGG16, VT gains +2.73 and +3.12 points over the matched control while saving 68% and 59% of SynOps relative to the full horizon, and every one of the eight thresholds lies above the fixed-timestep frontier. Both gains are three to five times the binomial half-width (±0.57 and ±0.64). These are the configurations in which VT works as advertised.

Moderate gains on adversarially trained models. On the adversarially trained fixed-T8 models, the gains are smaller: +1.37 and +2.89 on the two ResNet-18 runs, +0.89 on VGG16 and +0.28 on MobileNetV2. Savings are 35–42% for ResNet-18 and

VGG16. The two ResNet-18 runs, trained with identical settings, differ by a factor of two in gain, which is a direct warning that single-run gains of this size must be replicated before they are believed. Section 4.9 does so and finds that the gain on adversarially trained models is real.

Small gains where there is little to save. On MobileNetV2, VT exits late: the mean depth is 7.41 (clean) and 7.21 (adversarial), saving only 7–10% of SynOps. The clean MobileNetV2 has a large gain (+3.37), but its VT point is *above* the full-horizon accuracy (−0.42 below full), and it is matched against $T = 7$, where the fixed curve is flat (73.92% at $T = 6$, 74.71% at $T = 7$) before jumping to 77.66% at $T = 8$ (Table 4.2). This gain therefore reflects the irregular shape of MobileNetV2's fixed-timestep curve near the full horizon rather than an efficiency win, and we do not count it as evidence for VT. Gains must always be read together with depth and saving.

4.4.3 Where do inputs exit?

Figure 4.4 shows the distribution of exit timesteps at the VT-neutral operating points. The shape of these histograms explains the differences between configurations.

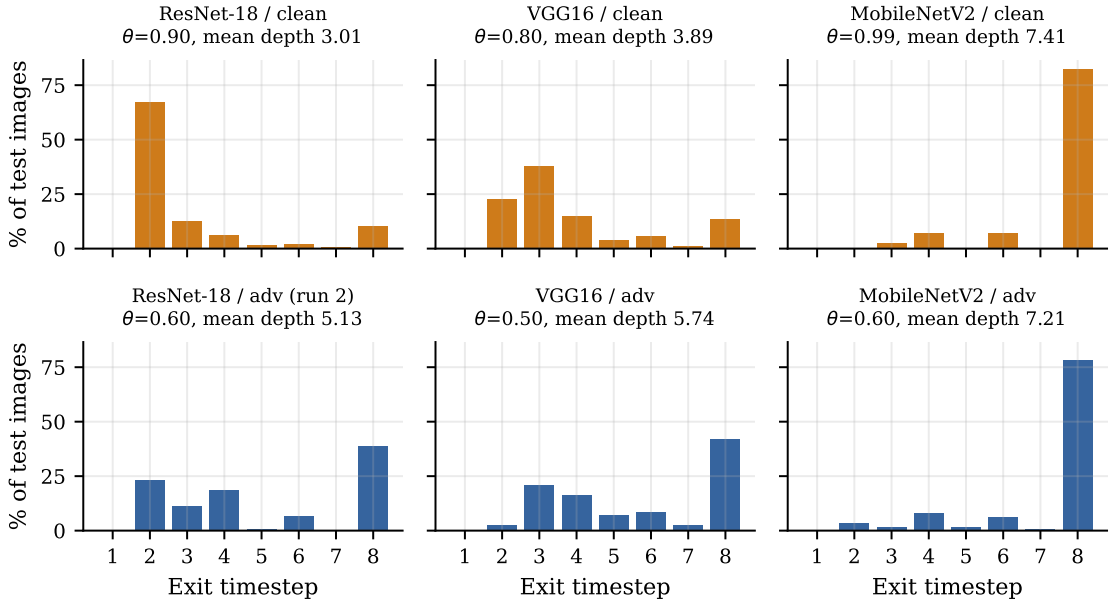


Figure 4.4: Distribution of exit timesteps at the VT-neutral operating points (test set). Clean-trained models (top) concentrate exits early, while adversarially trained models (bottom) send a large fraction of inputs to the full horizon.

For the clean-trained ResNet-18, 67% of test images exit at $t = 2$, the earliest step at which a fixed-T8 model has any information, and only 10% run to $t = 8$. The VT gain

comes from this bimodality: most images are already classified correctly and confidently at $t = 2$, and the compute saved on them is spent on the minority that need the full horizon. A fixed horizon of $T = 3$ spends the same total compute uniformly and so under-serves the hard images. VGG16 shows the same pattern with exits centred one step later, consistent with its slower fixed-timestep saturation.

Adversarially trained models are less confident: 39–42% of test images run to $t = 8$ on ResNet-18 and VGG16, and their softmax confidence rarely exceeds high thresholds, which is why their VT-neutral thresholds are low (0.5–0.6). MobileNetV2 sends about 80% of images to the full horizon in both regimes, leaving little room for savings. Exits at $t = 1$ never occur for fixed-T8 models (their $t = 1$ output is uninformative) but account for 7–8% of exits for mixed-T models.

4.4.4 Effect of the training schedule

Figure 4.5 compares the fixed-T8 and mixed-T ResNet-18 frontiers. Under adversarial training, mixed-T training raises the fixed-timestep curve at every horizon, and most at short horizons, which is exactly where VT drew its advantage. Under clean training, the fixed-timestep curve barely moves in accuracy but shifts to the right in compute, because the mixed-T network is less sparse. The VT curve, however, moves down: at $\theta = 0.95$ the mixed-T VT model reaches 84.15% at 2.02×10^8 SynOps, below the fixed horizon $T = 3$ (86.75% at 1.91×10^8).

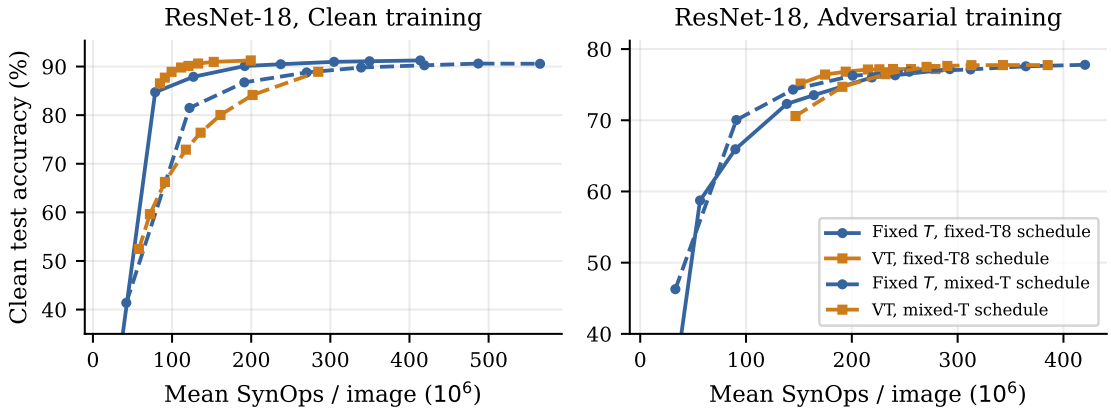


Figure 4.5: Effect of the training schedule on the ResNet-18 frontiers (test set). Solid lines show the fixed-T8 schedule and dashed lines the mixed-T schedule.

The gain of VT shrinks under mixed-T training: from +2.73 to +0.14 for clean training and from +2.89 (run 2, or +1.37 for run 1) to +0.68 for adversarial training, and only 3 and 5 of the 8 thresholds lie above the frontier. For the clean mixed-T model, no threshold came within $\delta = 1$ point of the full horizon on the calibration split, so the

most accurate threshold ($\theta = 0.99$) was used, 1.65 points below full.

The two regimes therefore lose the VT advantage for different reasons. Under adversarial training, the baseline gets stronger: a model trained across horizons is a much better short-horizon classifier, and per-input stopping adds little to it. Under clean training, the stopping signal gets worse: the mixed-T model is trained to commit to a prediction after one or two timesteps, and 43% of test images exit at $t \leq 2$ even at $\theta = 0.99$ (Figure 4.4 shows the fixed-T8 models for contrast), but its early confidence is a poorer indicator of correctness than that of the fixed-T8 model. In both cases, VT's advantage is largest when the fixed-horizon model has not been trained for short horizons but still produces confident and correct early predictions on easy inputs. This also means that VT's efficiency gain should be compared not only with a compute-matched horizon but with a *training-matched* one: a fair baseline for VT is a fixed-horizon model trained for the horizon at which it will be run. Mixed-T training is not free, however: on clean data it makes the network less sparse, and its VT points require more SynOps for the same accuracy than the fixed-T8 model's VT points do (Figure 4.5, left).

4.5 Stopping Rules

Does the conclusion depend on using the maximum softmax probability as the confidence score? Table 4.4 and Figure 4.6 compare the five stopping rules at approximately equal mean depth on the configurations where the reference depth is meaningful. (For the clean mixed-T ResNet-18 and the clean MobileNetV2, the stopping-rule comparison was calibrated to the depth of the degenerate max-gain operating point discussed in Section 4.6, so their rows are relegated to Appendix B.)

Table 4.4: Stopping-rule comparison (test accuracy % at mean exit depth in parentheses). Each rule's threshold was calibrated to match the depth of the max-softmax operating point.

Configuration	Fixed T=8	Max softmax	Margin	Entropy	Temp. softmax	Evidential
R18 clean	91.26	88.91 (2.39)	89.04 (2.38)	88.14 (2.34)	89.35 (2.59)	86.38 (2.19)
R18 adv (1)	73.67	71.48 (4.33)	73.31 (5.42)	72.24 (5.00)	73.61 (6.16)	67.99 (3.75)
R18 adv (2)	77.18	76.42 (5.13)	76.77 (5.46)	76.04 (5.19)	77.14 (6.25)	73.16 (4.62)
R18 adv mixT	77.79	77.50 (5.43)	77.47 (5.50)	77.41 (5.42)	77.37 (5.43)	76.13 (6.04)
VGG clean	88.20	86.99 (3.53)	86.99 (3.50)	86.11 (3.44)	87.39 (3.82)	82.54 (2.95)
VGG adv	65.52	65.21 (6.27)	65.40 (6.63)	64.46 (6.17)	65.45 (7.02)	62.22 (5.22)
MBv2 adv	51.44	50.92 (7.21)	51.30 (7.49)	50.91 (7.35)	51.49 (7.89)	49.48 (6.61)

Three findings emerge. First, **all four softmax-based rules lie above the fixed-timestep curve** on every configuration in Figure 4.6: the VT advantage is not spe-

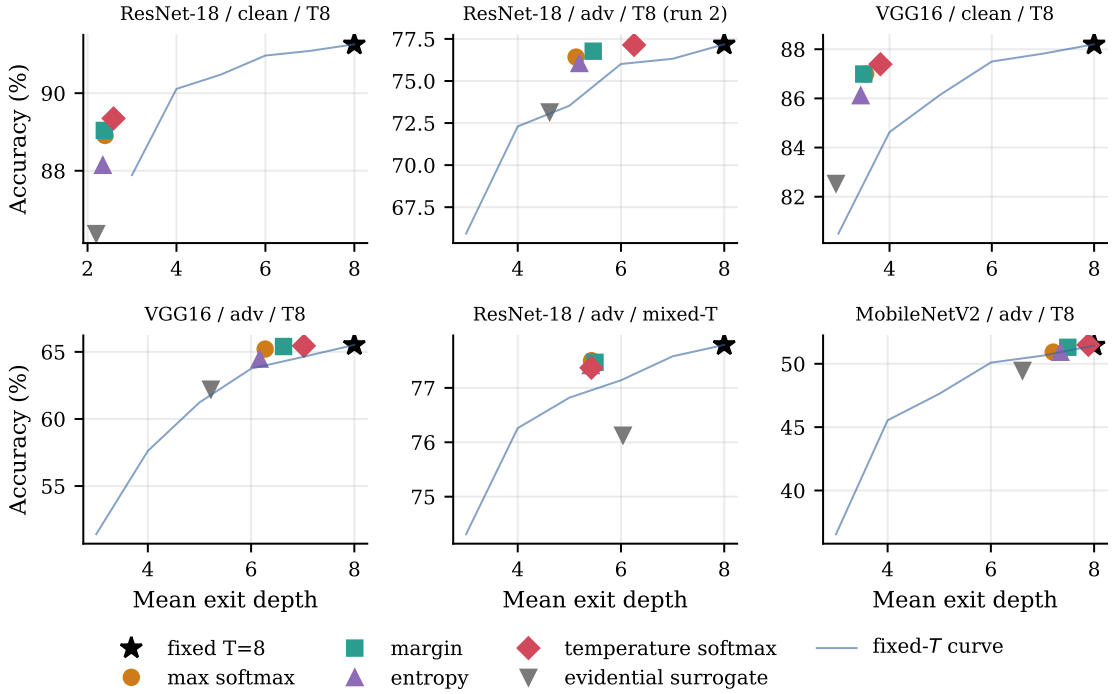


Figure 4.6: Stopping rules compared at matched exit depth. The thin line is the fixed-timestep curve, and points above it are better than a fixed horizon of the same depth.

cific to the max-softmax score. Second, max softmax, margin and entropy are within about one point of each other wherever they land at the same depth. Differences that look larger (for example 71.48% vs 73.31% on R18 adv (1)) come with a full timestep of extra depth, because the depth calibration on a discrete threshold grid is coarse. Temperature softmax tends to land deeper and correspondingly more accurately. Third, **the evidential surrogate is consistently the worst**: it is 1.4–4.5 points less accurate than max softmax, usually while exiting earlier, and lands on or only slightly above the fixed-timestep curve. On R18 adv mixT it lands deeper (6.04) yet less accurate, below the fixed curve. Its uncertainty mass saturates quickly for large logits, so it exits confidently on inputs that are not yet correctly classified. The choice among the softmax-based rules is therefore a second-order concern, and we use max softmax throughout.

4.6 RQ4: Sensitivity to the Selection Rule

4.6.1 Max-gain selection is biased

Table 4.5 compares the operating points chosen by the max-gain rule (used in our original experiments) with the VT-neutral points of Table 4.3, and Figure 4.7 visualises

the difference.

Table 4.5: Max-gain versus VT-neutral ($\delta = 1$) operating points (test set). Max-gain selection chooses the threshold with the largest calibration gain over its matched control.

	Max-gain selection					VT-neutral selection			
Configuration	θ	Depth	VT acc	Gain	Below full	θ	Depth	Gain	Below full
R18 clean	0.70	2.39	88.91	+4.19	2.35	0.90	3.01	+2.73	0.64
R18 adv (1)	0.50	4.33	71.48	+2.13	2.19	0.60	5.01	+1.37	0.84
R18 adv (2)	0.60	5.13	76.42	+2.89	0.76	0.60	5.13	+2.89	0.76
R18 clean mixT	0.60	1.38	59.63	+18.24	30.94	0.99	4.24	+0.14	1.65
R18 adv mixT	0.80	5.43	77.50	+0.68	0.29	0.80	5.43	+0.68	0.29
VGG clean	0.70	3.53	86.99	+6.49	1.21	0.80	3.89	+3.12	0.45
VGG adv	0.60	6.27	65.21	+1.46	0.31	0.50	5.74	+0.89	0.88
MBv2 clean	0.50	2.38	28.96	+11.36	48.70	0.99	7.41	+3.37	−0.42
MBv2 adv	0.60	7.21	50.92	+0.28	0.52	0.60	7.21	+0.28	0.52

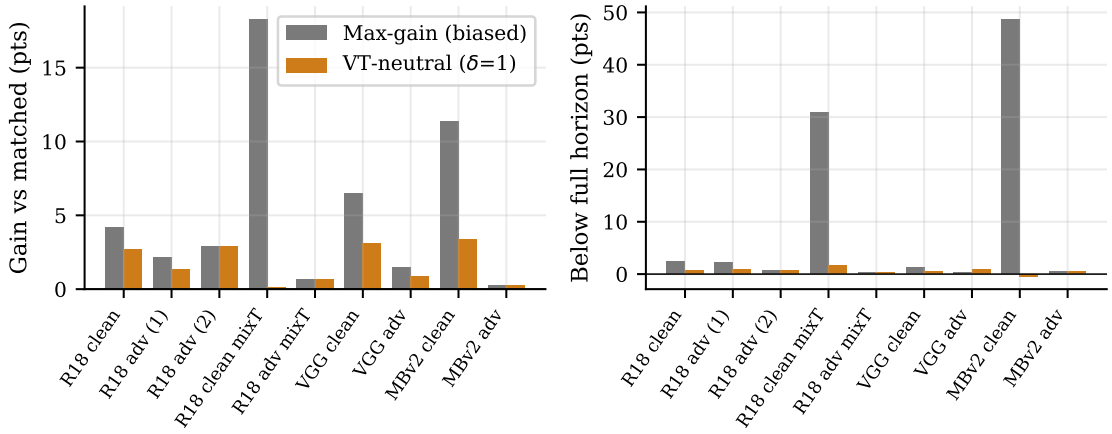


Figure 4.7: Effect of the selection rule. Left: gain over the compute-matched control. Right: accuracy given up relative to the full horizon. Max-gain selection inflates the gain on six of nine configurations, and on two it selects operating points 31–49 points below the full horizon.

On three configurations the two rules agree. On the other six, max-gain selection reports a larger gain, in five of them by choosing a lower threshold (earlier exits) than the VT-neutral rule. The inflation is moderate where both points are reasonable: +4.19 instead of +2.73 on R18 clean, +6.49 instead of +3.12 on VGG clean. On two configurations it is extreme. For the clean mixed-T ResNet-18, max-gain selection picks $\theta = 0.60$, where VT reaches only 59.63% with a mean depth of 1.38, and reports an +18.24 point gain over $T = 1$ (41.39%). For the clean MobileNetV2 it picks $\theta = 0.50$, where VT reaches 28.96%, and reports an +11.36 point gain over $T = 2$ (17.60%). Both operating points are useless in practice, at 31 and 49 points below the full-horizon ac-

curacy, and the large gains are comparisons between two bad models, one slightly less bad than the other.

The mechanism is simple. Near the start of the fixed-timestep curve, accuracy rises steeply with compute, so VT's ability to spend slightly more compute on some inputs buys a large accuracy difference relative to the matched horizon. Maximising the gain therefore drives the selection towards the steepest part of the fixed curve, which is also where both models are least accurate. The rule rewards VT for the very comparison it is being evaluated on.

In our original experiments this failure was caught for MobileNetV2 by inspection and overridden by hand with $\theta = 0.95$, and the attack experiments for that configuration use this threshold. It was not caught for the mixed-T ResNet-18, whose attack experiments therefore use the degenerate $\theta = 0.60$. The VT-neutral rule removes the need for such judgement calls.

4.6.2 Sensitivity to the tolerance

The VT-neutral rule has one parameter, the tolerance δ. Table 4.6 and Figure 4.8 report the gain at $\delta \in \{0.5, 1, 2\}$.

Table 4.6: Gain over the compute-matched control (points) under VT-neutral selection with tolerance δ.

Configuration	$\delta = 0.5$	$\delta = 1$	$\delta = 2$
R18 clean	+1.15	+2.73	+2.24
R18 adv (1)	+2.03	+1.37	+1.37
R18 adv (2)	+2.89	+2.89	+1.62
R18 clean mixT	+0.14	+0.14	+0.14
R18 adv mixT	+0.68	+0.68	−0.35
VGG clean	+3.31	+3.12	+6.49
VGG adv	+1.46	+0.89	+0.89
MBv2 clean	+3.37	+3.37	+3.37
MBv2 adv	−0.06	+0.28	+0.28

The sign of the gain is stable: 25 of the 27 entries are positive, and the two negative entries (-0.35 and -0.06) are on the two configurations whose gains were already within noise. The magnitude is not stable: because the threshold grid is discrete and the fixed horizon is an integer, changing δ can move the selected threshold by one grid step and the matched horizon by one timestep, which can change the gain by one to three points (from +1.15 to +2.73 on R18 clean and from +3.12 to +6.49 on VGG clean). The qualitative conclusions of Section 4.4 hold for every δ: clear gains

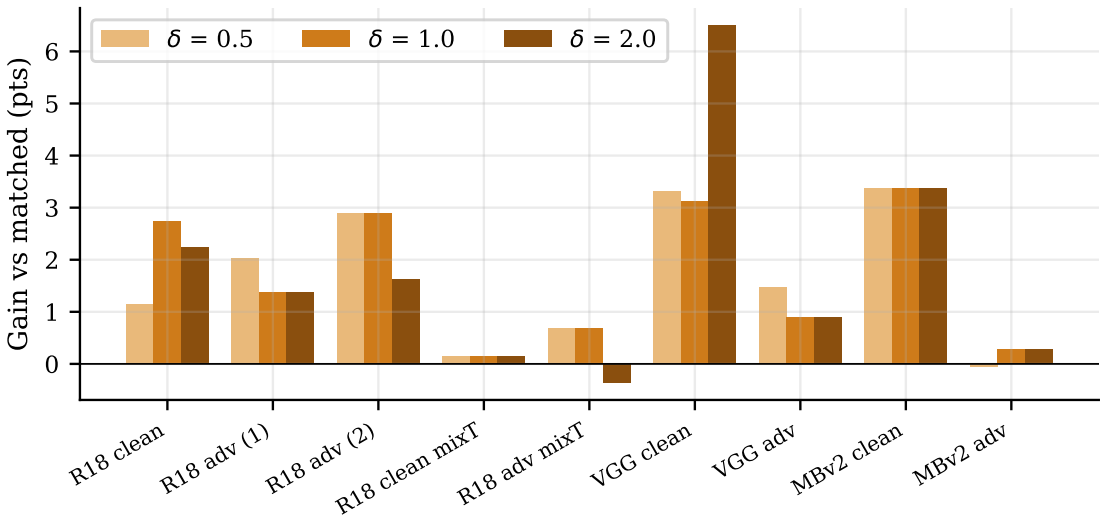


Figure 4.8: Sensitivity of the VT-neutral gain to the tolerance δ.

on clean fixed-T8 ResNet-18 and VGG16, smaller gains under adversarial training and negligible gains under mixed-T training. However, but single-point gain values should be read with roughly ± 1.5 points of selection uncertainty in addition to evaluation noise.

4.7 RQ2: Adversarial Robustness

We now ask how VT changes robustness. This section reports the single-run results for every configuration, and Section 4.9 then tests which of them replicate. All attacks are white-box with $\epsilon = 8/255$, and the attacker differentiates through the exit-step logits of the VT model (Section 3.10). As explained in Section 3.8, the attacked VT thresholds are those chosen by the original max-gain rule (with the MobileNetV2 override to $\theta = 0.95$). For the adversarially trained models these coincide with or neighbour the VT-neutral thresholds.

4.7.1 Clean-trained models have no robustness to lose

Table 4.7 shows that every clean-trained model collapses under attack, with or without VT. ResNet-18 and VGG16 are at or near 0% under both PGD-20 and AutoAttack, and MobileNetV2 retains about 5%. The differences between fixed and VT inference are a fraction of a point (the 2.25% of the clean mixed-T ResNet-18 under AutoAttack is at its degenerate $\theta = 0.60$ point, where clean accuracy is only 59.63%). Without adversarial training, VT neither helps nor hurts robustness, because there is no robustness to help or hurt. The rest of this section concerns adversarially trained models.

Table 4.7: Single-run robust accuracy (%) of full-horizon ($T = 8$) and VT inference under PGD-20 and AutoAttack ($\epsilon = 8/255$). θ: attacked threshold. n: number of attacked images. The run-1 adversarial ResNet-18 is omitted (see Section 4.11).

			PGD-20			AutoAttack		
Configuration	θ	n	T=8	VT	Δ	T=8	VT	Δ
R18 clean	0.70	10,000	0.00	0.02	+0.02	0.06	0.02	−0.04
R18 clean mixT	0.60	2,000	0.00	0.15	+0.15	0.10	2.25	+2.15
VGG clean	0.70	2,000	0.05	0.10	+0.05	0.00	0.00	+0.00
MBv2 clean	0.95	2,000	5.10	4.95	−0.15	5.25	5.40	+0.15
R18 adv (2)	0.60	2,000	46.10	48.70	+2.60	28.85	24.15	−4.70
R18 adv mixT	0.80	2,000	40.85	41.50	+0.65	29.45	25.55	−3.90
VGG adv	0.60	2,000	45.90	44.45	−1.45	10.75	8.55	−2.20
MBv2 adv	0.60	2,000	35.60	34.75	−0.85	27.60	27.55	−0.05

4.7.2 PGD and AutoAttack disagree

For the adversarially trained models, the two attacks tell different stories (Figure 4.9). Under PGD-20, VT is within ±2.6 points of the full-horizon model and is *better* on two of four configurations. A reader who saw only PGD would conclude that VT costs nothing in robustness. Under AutoAttack, VT is worse than the full horizon on every adversarially trained configuration, by up to 4.7 points (and tied, at −0.05, on MobileNetV2, whose VT point runs 7.2 of 8 timesteps).

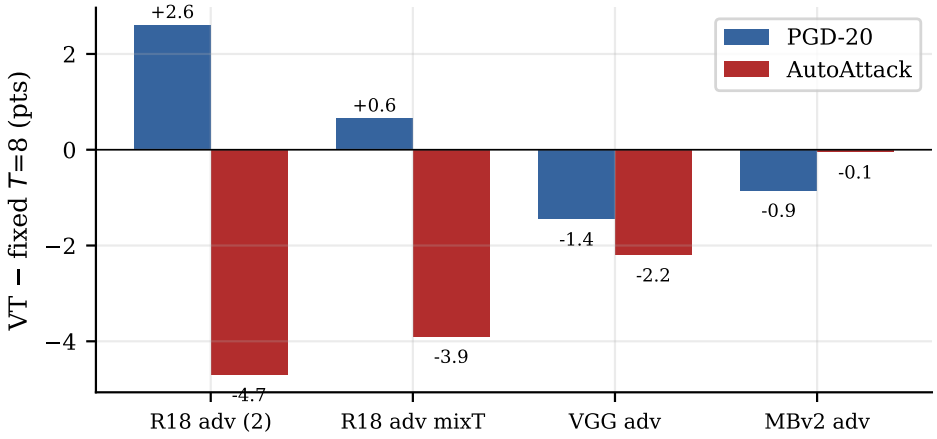


Figure 4.9: Robust accuracy of VT minus that of the full-horizon model ($T = 8$), single run, adversarially trained models. PGD-20 suggests that VT is as robust as the full horizon, but AutoAttack shows that it is less robust.

The gap between PGD and AutoAttack is itself large: 11–17 points for ResNet-18, 8 points for MobileNetV2 and 35 points for VGG16 at the full horizon. Gaps of this size indicate that PGD-20 is far from the worst case for these models. For VGG16 in particular, its PGD robustness is largely illusory. This matches the conclusion of

Wang et al. [66] that SNN robustness is often over-estimated by standard gradient attacks, and it means that *any* statement about VT and robustness made on the basis of PGD alone should be treated with suspicion. The direction of VT's effect also differs between the attacks: the gradient-free Square attack and the adaptive-step APGD in AutoAttack are evidently able to exploit something that PGD misses, and the multi-seed experiments below locate what it is.

4.7.3 What attacks do to exit depth

Attacks change not only VT's predictions but also when it stops. On adversarially trained models, untargeted PGD *increases* the mean exit depth: from 5.13 to 5.86 timesteps on R18 adv (2), from 6.27 to 6.69 on VGG adv, from 7.21 to 7.45 on MBv2 adv and from 5.43 to 6.35 on R18 adv mixT. On clean-trained models it does the opposite: the attacked inputs of R18 clean exit at a mean depth of 2.00 (from 2.39 on clean inputs) and those of VGG clean at 2.40 (from 3.53). The adversary makes clean-trained models confidently wrong at the first informative timestep, while adversarially trained models become less confident and keep integrating.

Figure 4.10 shows why for an adversarially trained ResNet-18 (1,000 test images). Under PGD-20 the running prediction is less accurate at every timestep, and its mean confidence is lower: at $t = 8$ the mean max-softmax confidence is 0.61 on attacked inputs against 0.68 on clean ones, so attacked inputs cross a threshold of 0.60 later. Robust accuracy rises from 36% at $t = 2$ to 49% at $t = 8$, a 12.5-point gain from integrating six more timesteps, proportionally as large as the corresponding clean gain (from 57% to 78%). This is the central mechanism behind the robustness results: **for an adversarially trained SNN, robustness accumulates over timesteps**. An inference mode that runs fewer timesteps has, on average, less of it.

4.8 RQ3: The Exit Gate as an Attack Surface

4.8.1 Wrong early exit

The wrong-early-exit attack (Equation (3.13)) explicitly tries to make inputs exit early with a wrong label. Table 4.8 compares it with untargeted PGD on the same VT operating points.

The gate-aware objective is not a stronger misclassification attack than ordinary PGD. On clean-trained models both attacks reach about 0%, and untargeted PGD *already* produces wrong early exits: attacked inputs leave at the earliest informative timestep.

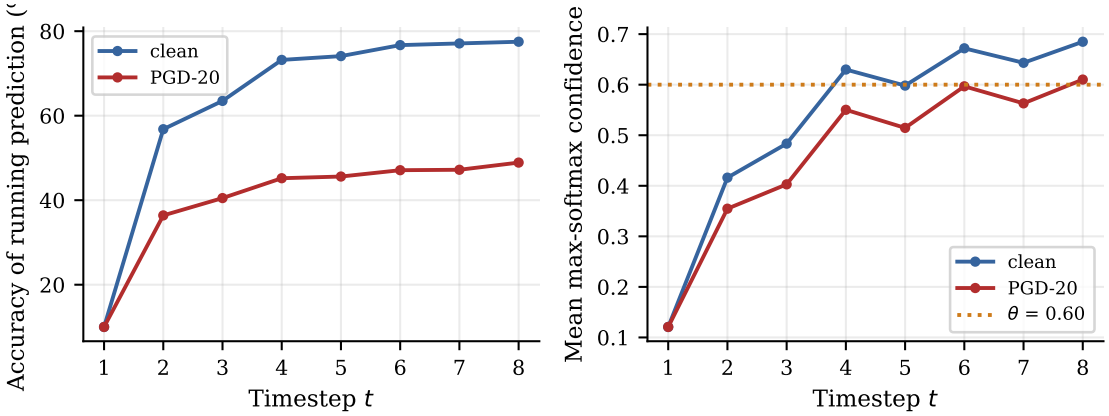


Figure 4.10: Accuracy (left) and mean max-softmax confidence (right) of the running prediction at each timestep for an adversarially trained fixed-T8 ResNet-18, on clean and PGD-20 inputs (1,000 test images). Robust accuracy keeps rising with the number of timesteps, and attacked inputs are less confident and cross the threshold later.

Table 4.8: Wrong-early-exit attack versus untargeted PGD-20 at the attacked VT threshold (robust accuracy %, with mean exit depth in parentheses). $n = 2{,}000$ except for R18 clean ($n = 10{,}000$).

Configuration	θ	Untargeted PGD-20	Wrong early exit	Δ acc
R18 clean	0.70	0.02 (2.00)	0.00 (2.00)	−0.02
R18 clean mixT	0.60	0.15 (1.02)	2.60 (1.18)	+2.45
VGG clean	0.70	0.10 (2.40)	0.00 (2.45)	−0.10
MBv2 clean	0.95	4.95 (7.17)	4.05 (6.95)	−0.90
R18 adv (2)	0.60	48.70 (5.86)	45.95 (5.62)	−2.75
R18 adv mixT	0.80	41.50 (6.35)	42.55 (6.27)	+1.05
VGG adv	0.60	44.45 (6.69)	46.35 (6.62)	+1.90
MBv2 adv	0.60	34.75 (7.45)	33.35 (7.42)	−1.40

On adversarially trained models the two attacks are within ±3 points of each other (comparable to the ±2 point evaluation noise) and neither consistently wins. The wrong-early-exit attack shortens the mean depth by at most a quarter of a timestep. The adversarially trained models' low confidence under attack (Figure 4.10) makes it hard for any attack to force a confident early exit within the ϵ budget. We conclude that, within our threat model, the exit gate does not open a new route to misclassification beyond what standard attacks already find.

4.8.2 Compute overuse

The compute-overuse attack (Equation (3.14)) has the opposite goal: keep the prediction correct but delay the exit. Table 4.9 and Figure 4.11 report the inflation of mean exit depth it achieves.

Table 4.9: Compute-overuse attack at the attacked VT threshold. Clean depth: mean exit depth on clean test inputs. Inflation: increase in mean exit depth under attack. Headroom: inflation as a fraction of the timesteps available, 8 − clean depth. Kept: accuracy on attacked inputs (the attack aims to preserve the label).

Configuration	θ	Clean depth	Attacked depth	Inflation	Headroom	Kept (%)
R18 clean	0.70	2.39	3.58	+1.19	21%	60.6
R18 clean mixT	0.60	1.38	1.55	+0.17	3%	81.6
VGG clean	0.70	3.53	4.70	+1.17	26%	72.1
MBv2 clean	0.95	6.56	7.71	+1.15	80%	97.2
R18 adv (1)	0.50	4.33	4.70	+0.37	10%	86.5
R18 adv (2)	0.60	5.13	5.40	+0.27	10%	92.0
R18 adv mixT	0.80	5.43	6.08	+0.65	25%	95.0
VGG adv	0.60	6.27	6.31	+0.04	2%	81.0
MBv2 adv	0.60	7.21	7.26	+0.05	7%	67.1

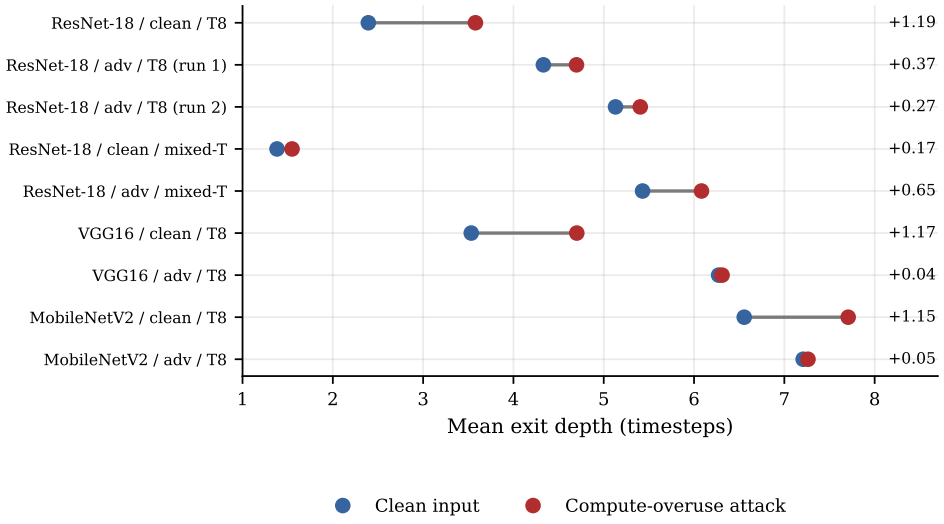


Figure 4.11: Mean exit depth on clean inputs (blue) and under the label-preserving compute-overuse attack (red) at the attacked VT threshold. Numbers on the right give the inflation in timesteps.

Clean-trained fixed-T8 models are highly vulnerable. On all three backbones, the attack raises the mean exit depth of the clean-trained fixed-T8 model by more than one full timestep (1.15–1.19), using 21–80% of the available headroom. For the clean ResNet-18 this means a 50% increase in exit depth (2.39 to 3.58 timesteps), eroding a substantial part of the compute saving that motivated VT. The attack preserves the label on 61–97% of inputs. A perturbation that is invisible to accuracy monitoring yet substantially increases energy consumption is precisely the threat that motivates sponge and slowdown attacks [27], [58], and VT SNNs are susceptible to it.

Adversarial training largely removes the vulnerability. On adversarially trained fixed-T8 models, inflation falls to 0.04–0.37 timesteps (2–10% of headroom). This is not only a ceiling effect from their higher clean depth: as a fraction of available headroom the inflation is two (ResNet-18) to more than ten (VGG16, MobileNetV2) times smaller than for clean-trained models. Adversarial training flattens the network's confidence response to small input perturbations in general, and that protects the exit gate as a side effect. The exception is the adversarially trained mixed-T ResNet-18, whose inflation (0.65 timesteps, 25% of headroom) is comparable to that of clean-trained models. The mixed-T schedule, which trains the network to commit at short horizons, seems to make its early confidence more malleable.

Clean mixed-T. The low inflation of the clean mixed-T ResNet-18 (0.17) is an artefact of its degenerate attacked threshold ($\theta = 0.60$, depth 1.38), at which most inputs exit at $t \leq 2$ regardless, so it should not be read as robustness.

4.8.3 Summary of the attack surface

The exit gate creates one new practical vulnerability and not the one usually feared. It does not make misclassification easier, but it makes *compute* attackable: a VT model's energy cost becomes a function of the input that an adversary can raise while leaving accuracy intact. The vulnerability is severe for clean-trained models and largely mitigated by adversarial training. A deployment that relies on VT for an energy or latency budget should therefore either use adversarially trained models or enforce a hard compute cap.

4.9 Multi-Seed Replication

The single-run results contain both effects that may be real and effects that may be noise. The two identical ResNet-18 adversarial runs differ by 3.5 points of full-horizon accuracy and by a factor of two in VT gain (Table 4.3), so single runs cannot settle which is which. This section reports the three-seed replication for adversarially trained ResNet-18 and VGG16.

4.9.1 Per-seed results

Table 4.10 lists the results for each of the six checkpoints. All three inference modes are evaluated on the same 1,000 class-balanced test images.

Table 4.10: Multi-seed results per checkpoint (accuracy %, $n = 1{,}000$). Full: $T = 8$. Matched: compute-matched fixed horizon ($T^* = 5$ for ResNet-18, $T^* = 6$ for VGG16). VT: selected threshold.

			Clean			PGD-20			AutoAttack		
Backbone	Seed	θ	Full	Match	VT	Full	Match	VT	Full	Match	VT
ResNet-18	0	0.5	78.6	74.7	76.0	56.3	56.4	57.0	32.0	23.4	24.3
	1	0.5	77.8	72.8	74.9	50.7	49.2	51.0	31.7	24.6	29.9
	2	0.6	74.7	72.3	75.3	47.3	48.9	48.3	24.8	20.2	22.2
VGG16	0	0.7	66.9	64.4	68.5	43.9	43.6	44.6	18.3	19.9	16.4
	1	0.5	66.9	64.3	66.1	46.0	47.2	45.6	23.1	21.4	15.3
	2	0.6	67.2	64.3	66.8	47.5	46.0	46.2	20.5	14.5	18.0

The checkpoints vary widely. Full-horizon AutoAttack accuracy ranges from 24.8% to 32.0% across ResNet-18 seeds and from 18.3% to 23.1% across VGG16 seeds, and seed 2 of ResNet-18 is about 3 points less accurate on clean data than seeds 0 and 1. Differences of this size between checkpoints are larger than most of the single-run effects in Section 4.7, which is why the analysis below is based on paired, within-checkpoint differences.

4.9.2 Paired tests

Table 4.11 reports the paired differences pooled over the six checkpoints, and Figure 4.12 shows every checkpoint's difference.

Table 4.11: Paired differences (VT minus reference, points) pooled over six adversarially trained checkpoints (three ResNet-18, three VGG16). $+/n$: checkpoints with a positive difference. p_t: one-sample t-test, 5 degrees of freedom. p_{sign}: exact two-sided sign test.

Reference	Metric	Mean $\pm$ SD	$+/n$	t	p_t	p_{sign}
Matched fixed T^*	Clean	$+2.47 \pm 0.99$	6/6	6.11	**0.002**	**0.031**
	PGD-20	$+0.23 \pm 1.20$	4/6	0.48	0.655	0.688
	AutoAttack	$+0.35 \pm 4.33$	4/6	0.20	0.851	0.688
Full horizon $T = 8$	Clean	-0.75 ± 1.76	2/6	-1.04	0.345	0.688
	PGD-20	$+0.17 \pm 0.87$	4/6	0.47	0.657	0.688
	AutoAttack	-4.05 ± 2.88	0/6	-3.44	**0.018**	**0.031**

Two results replicate on every checkpoint.

VT's clean efficiency gain replicates. VT is more accurate than its compute-matched control on clean data on all six checkpoints, by $+2.47 \pm 0.99$ points ($t(5) = 6.11$,

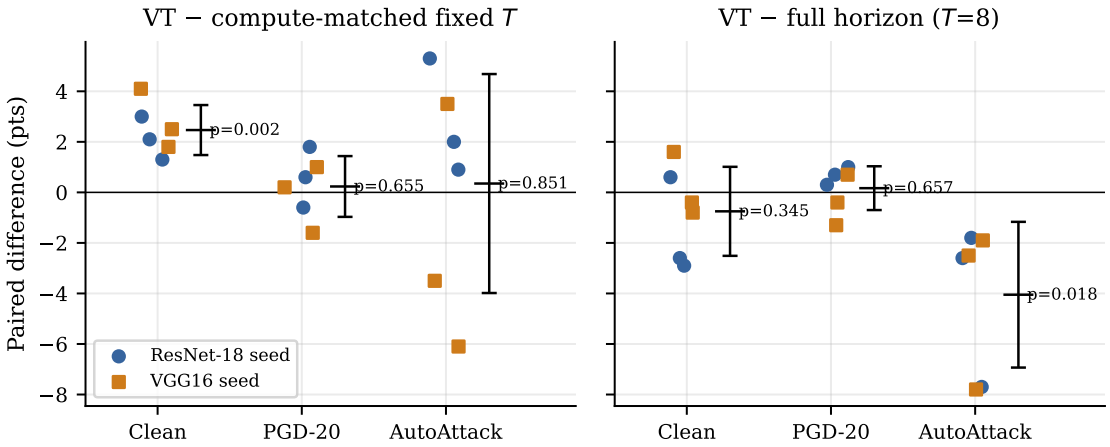


Figure 4.12: Paired differences for each adversarially trained checkpoint. Left: VT minus the compute-matched fixed horizon. Right: VT minus the full horizon. Black bars: mean ± one standard deviation, with the t-test p-value.

$p = 0.002$). The effect holds on each backbone separately ($+2.13 \pm 0.85$ for ResNet-18, $+2.80 \pm 1.18$ for VGG16). On clean data, VT is statistically indistinguishable from the full horizon (-0.75, $p = 0.35$) while running 18–39% fewer timesteps on average. This answers RQ1 for adversarially trained models: the single-run gains of Table 4.3 were not noise.

VT's AutoAttack cost relative to the full horizon replicates. Under AutoAttack, VT is less robust than the full-horizon model on all six checkpoints, by -4.05 ± 2.88 points ($t(5) = -3.44$, $p = 0.018$, sign test $p = 0.031$). The cost is the same on both backbones (-4.03 for ResNet-18, -4.07 for VGG16).

The comparisons that do not replicate are equally informative.

At equal compute, VT is as robust as a fixed horizon. Against its matched control, VT's robustness difference is $+0.23 \pm 1.20$ under PGD-20 and $+0.35 \pm 4.33$ under AutoAttack, with four of six checkpoints positive in each case. There is no evidence that VT is more *or* less robust than a fixed-horizon SNN that uses the same compute. The large AutoAttack variance is driven by VGG16, whose matched-control robustness varies between 14.5% and 21.4% across seeds.

PGD hides the cost. Under PGD-20, VT is indistinguishable from the full horizon ($+0.17 \pm 0.87$), and on ResNet-18 it is even slightly better on all three seeds. The AutoAttack deficit of 4 points is invisible to PGD.

4.9.3 Interpretation: robustness follows compute

Taken together, the multi-seed results say that *VT's robustness cost is a compute cost*. VT runs adversarially trained models for about five (ResNet-18) to six and a half (VGG16) timesteps on average instead of eight, and under AutoAttack it loses about four points relative to the full horizon. The compute-matched fixed horizon, which runs the same number of timesteps for every input, loses a similar amount: its AutoAttack accuracy is on average 4.4 points below the full horizon on the same checkpoints (Table 4.10). VT and the matched horizon are, within the precision of our experiment, equally robust. What VT gives up is the robustness that the skipped timesteps would have provided, which is exactly what the confidence trajectories of Figure 4.10 predict: for adversarially trained SNNs, robust accuracy keeps rising with the number of timesteps integrated.

This interpretation reconciles the efficiency and robustness results. VT is a better way to spend a given compute budget on clean data, because it allocates timesteps to the inputs that need them. Under a strong attack, every input is hard, the confidence of attacked inputs is low, and the advantage of adaptive allocation disappears. What remains is the total compute spent, and VT spends less of it than the full horizon.

4.9.4 Attack-seed variance

To verify that the attack itself is not a significant source of variance, PGD-20 and AutoAttack were repeated with three attack seeds on one checkpoint per backbone. Table B.14 in Appendix B lists the results. The standard deviation across attack seeds is at most 1.74 points and usually below one point, much smaller than the variance across training seeds. The conclusions above are therefore limited by training variance, not by attack randomness.

4.10 Discussion

4.10.1 What VT is, and what it is not

The results support a precise statement of what variable timing does. On clean data, VT is a genuinely better allocation of compute than truncating every input at the same horizon: at equal measured SynOps it is 1–3 points more accurate on the configurations where there is something to save, it can reach within a point of the full horizon at 35–68% lower compute, and the advantage replicates across seeds. Its benefit is largest for networks trained at the full horizon only and shrinks for networks trained

across horizons, so part of what VT exploits is the poor short-horizon performance of fixed-T8-trained networks.

VT is *not* a robustness mechanism, positively or negatively, at equal compute. Our data do not support the hypothesis that adaptive exit makes SNNs more fragile, nor the hypothesis that it makes them more robust. They support a simpler one: robustness of adversarially trained SNNs grows with the number of timesteps integrated, and VT integrates fewer.

4.10.2 Why PGD-only evaluation would have misled us

Had we evaluated only with PGD-20, we would have concluded that VT saves 40% of compute at no cost in robustness, which is a strong and false claim. Two properties of the setting conspire to produce this. First, the PGD–AutoAttack gap is large for SNNs (8–35 points at the full horizon in our models), so PGD robustness numbers are far from the worst case. Second, VT's robustness cost is smaller than that gap, so it is hidden inside it. Any evaluation of an efficiency method for SNNs that reports robustness should use AutoAttack or an equally strong adaptive attack, compare with a compute-matched baseline, and replicate across seeds.

4.10.3 Compute as an attack surface

The compute-overuse results suggest that energy should be treated as a security property of VT SNNs. A clean-trained VT model that is deployed because it saves 60–70% of compute can be made to run roughly 50% more timesteps by an imperceptible perturbation, without any change in its accuracy. Adversarial training reduces this to a few percent of the headroom, but at a 14–26 point cost in clean accuracy. A practical middle ground would be to cap the compute any single input may use, or to monitor the distribution of exit depths at run time for signs of manipulation.

4.10.4 Recommendations for evaluating VT

From these results we draw five recommendations for future evaluations of variable timing and similar early-exit methods in SNNs:

1. Compare every VT operating point with the fixed horizon of *equal measured compute*, using sample-exact accounting, not only with the full horizon.

2. Select operating points with a rule that does not reward the comparison being made (for example, the VT-neutral rule), on a split separate from the test set,

and report depth and saving alongside every gain.

3. Compare against a fixed-horizon model that was trained for short horizons (for example with a mixed-T schedule), not only one trained at the full horizon.

4. Evaluate robustness with AutoAttack or an equally strong adaptive attack, with gradients taken through the deployed exit rule, and report the compute-matched and full-horizon comparisons separately.

5. Report compute under attack, including a dedicated compute-overuse attack, and replicate key comparisons across training seeds with paired tests.

4.11 Threats to Validity

4.11.1 Internal validity

Operating points for attacks. The attack experiments were run at the thresholds chosen by the max-gain rule (Section 3.8). For adversarially trained models these are the VT-neutral thresholds or their neighbours, and the attack files include the neighbouring thresholds, whose results are consistent with the selected ones (Appendix B). For clean-trained models the thresholds differ, but all clean-trained robust accuracies are near zero. The clean mixed-T ResNet-18 attack results are at a degenerate operating point and are interpreted only as such.

Discrete matching. Fixed horizons are integers and the threshold grid has eight values, so the matched control never uses exactly the same SynOps as the VT point. At the VT-neutral points the residual mismatch is within $\pm 11\%$ of the matched SynOps. VT uses slightly more compute than its control on most configurations and 9–10% less on VGG16 (Appendix B). The gain on VGG16 is therefore, if anything, understated. The tolerance analysis (Section 4.6) shows that this discreteness contributes roughly ± 1.5 points of uncertainty to individual gains, but does not change their sign on configurations with clear gains.

Withheld measurements. The full-test attack evaluation of the first adversarially trained ResNet-18 (run 1) gave 20.96% (fixed $T = 8$) and 23.12% (VT) under PGD-20, and 7.57% and 4.14% under AutoAttack, on all 10,000 test images. These numbers cannot be reconciled with the same checkpoint's evaluation on a 1,000-image subset (44.1% and 22.2% for the full horizon), nor with the independently trained run 2 (46.10% and 28.85%), under identical recorded attack parameters. A discrepancy of

this size cannot come from subsampling, so we treat the full-test numbers as a measurement error and exclude them. This checkpoint's clean-data and compute-overuse results are unaffected and are reported.

4.11.2 External validity

One dataset, one horizon. All results are on CIFAR-10 with $T_{\max} = 8$ and direct encoding. Datasets on which accuracy saturates more slowly, longer horizons, or event-based inputs with genuine temporal structure could change the size of both the efficiency gain and the robustness cost.

Limited multi-seed coverage. The multi-seed replication covers adversarially trained fixed-T8 ResNet-18 and VGG16 with three seeds each. Clean-trained models, mixed-T models and MobileNetV2 (whose multi-seed run timed out) were not replicated. With six checkpoints, the paired tests can detect consistent effects of a few points but not small ones, so the non-significant comparisons should be read as "no detectable difference", not as "no difference".

The multi-seed selection rule. The multi-seed experiments selected operating points with the max-gain rule and saved only those points, so they could not be re-selected with the VT-neutral rule. The selected points are within about three points of the full horizon and use the same matched horizons as the single-run VT-neutral points, so we do not expect this to change the conclusions, but the clean-data gain could be slightly inflated.

Compute metric. SynOps is a proxy. It does not capture memory traffic, membrane-state updates or the control overhead of per-sample early exit, and real neuromorphic hardware may batch inputs in ways that reduce the benefit of per-sample stopping [11]. Our comparisons are relative (VT versus fixed horizons on the same network), which reduces but does not remove this concern.

4.11.3 Construct validity

Attack strength. AutoAttack is the strongest standard attack we used, but it was designed for ANNs. SNN-specific adaptive attacks [66] might lower robust accuracy further. Since our main robustness conclusion is relative (VT versus fixed horizons under the same attack), a stronger attack would most likely enlarge rather than remove the PGD–AutoAttack discrepancy we report. The gate-specific attacks were de-

signed by us and may not be optimal. In particular, the wrong-early-exit result in particular shows only that *our* attack did not outperform PGD.

4.12 Summary

On clean data, variable timing is more accurate than fixed-timestep inference at equal compute on every configuration we trained, and the gain replicates across seeds (+2.47 points, $p = 0.002$). It is largest for networks trained only at the full horizon. Under attack, VT is as robust as a fixed horizon of equal compute, but under AutoAttack it is consistently less robust than the full horizon (-4.05 points, 0 of 6 seeds positive), because robustness accumulates with the timesteps it skips. PGD-20 hides this cost. The exit gate does not make misclassification easier, but it makes compute attackable: a label-preserving attack inflates the exit depth of clean-trained models by more than a timestep, a vulnerability that adversarial training largely removes. The way operating points are selected can inflate reported gains dramatically. The next chapter draws these findings together.

Chapter 5

Conclusion

This chapter closes the thesis. Section 5.1 restates the problem, Section 5.2 answers the research questions, and Section 5.3 revisits the objectives. Section 5.4 summarises the contributions and Section 5.5 their implications. Section 5.6 states the limitations, Section 5.7 outlines future work, and Section 5.8 offers concluding remarks.

5.1 Restatement of the Problem

Spiking neural networks process each input over several timesteps, and their cost grows with every timestep they run. Variable timing lets each input stop as soon as the network's accumulated prediction is confident, promising lower average cost at little loss in accuracy. Before this thesis, it was not known whether that promise survives a comparison against a fixed-horizon SNN of equal compute, how a confidence-gated exit changes adversarial robustness and whether it can itself be attacked, or whether such conclusions survive changes in operating-point selection, attack strength and training seed. We studied these questions with deterministic LIF SNNs on CIFAR-10, across three backbones, two training regimes and two timestep schedules, under a compute-matched protocol with a VT-neutral selection rule, strong and gate-specific attacks, and a multi-seed replication.

5.2 Answers to the Research Questions

Table 5.1 summarises the answers, and the paragraphs below expand on them.

Table 5.1: Summary of the answers to the research questions.

RQ	Answer	Key evidence
RQ1 Efficiency	Yes. At equal compute, VT is more accurate than fixed-timestep inference on all nine configurations; the gain replicates across seeds. It is large for fixed-T8-trained models and small for mixed-T-trained ones.	Table 4.3; Table 4.11: $+2.47 \pm 0.99$, $p = 0.002$
RQ2 Robustness	At equal compute, VT is as robust as a fixed horizon. Relative to the full horizon, it loses AutoAttack robustness in proportion to the timesteps it skips. PGD-20 hides this cost.	Table 4.11: +0.35 ($p = 0.85$) vs matched; -4.05 ($p = 0.018$, 0/6) vs full
RQ3 Attack surface	The gate does not ease misclassification but makes compute attackable: a label-preserving attack inflates exit depth by over one timestep on clean-trained models; adversarial training largely removes this.	Table 4.8; Table 4.9
RQ4 Reliability	Efficiency conclusions are robust in sign but not in magnitude; max-gain selection can inflate gains up to +18 points at useless operating points; single-run robustness comparisons did not replicate.	Table 4.5; Table 4.6; Section 4.11

RQ1: Is VT more accurate than a fixed horizon at equal compute? Yes. At the VT-neutral operating points, VT beats its compute-matched fixed horizon on all nine configurations, by +0.14 to +3.37 points. On clean-trained fixed-T8 ResNet-18 and VGG16 the gains are clear (+2.73 and +3.12) and come with 68% and 59% SynOps savings relative to the full horizon at less than one point of accuracy loss. On adversarially trained models the single-run gains are smaller and variable, but the three-seed replication on ResNet-18 and VGG16 confirms them: $+2.47 \pm 0.99$ points pooled, positive on all six checkpoints. The gain shrinks to +0.14 (clean) and +0.68 (adversarial) when the network is trained with a mixed-T schedule, because the mixed-T fixed-horizon baseline is stronger at short horizons or the mixed-T confidence signal is less informative. VT's advantage is therefore real, but it is largest against fixed-horizon models that were never trained for short horizons.

RQ2: How does VT change robustness? Clean-trained models collapse under attack regardless of inference mode. For adversarially trained models, the answer depends on the reference and the attack. Against the compute-matched fixed horizon, VT shows no detectable robustness difference under either PGD-20 (+0.23) or AutoAttack (+0.35). Against the full horizon, VT is indistinguishable under PGD-20

(+0.17) but consistently less robust under AutoAttack (−4.05, 0 of 6 checkpoints positive, $p = 0.018$). The matched horizon loses a similar 4.4 points relative to the full horizon. VT's robustness cost is therefore a compute cost: robustness of adversarially trained SNNs accumulates over timesteps, and VT runs fewer of them. Because the PGD–AutoAttack gap of these models (8–35 points) is larger than the cost, a PGD-only evaluation would wrongly conclude that VT is free.

RQ3: Does the exit gate create new attacks? Partially. A wrong-early-exit attack designed to force confident wrong exits was not stronger than standard PGD on any adversarially trained model. On clean-trained models, ordinary PGD already causes wrong exits at the earliest informative timestep. A label-preserving compute-overuse attack, however, raised the mean exit depth of clean-trained fixed-T8 models by 1.15–1.19 timesteps while keeping 61–97% of predictions correct, an energy-latency attack that fixed-horizon inference does not admit. Adversarial training reduced the inflation to 0.04–0.37 timesteps (2–10% of the available headroom), except for the mixed-T model (0.65 timesteps).

RQ4: How reliable are these conclusions? The sign of the clean efficiency gain is robust to the tolerance of the VT-neutral rule (25 of 27 settings positive) and to the stopping rule (all softmax-based rules beat the fixed horizon), but its magnitude moves by one to three points with the discrete choice of threshold and matched horizon. The max-gain selection rule used in our original experiments, and natural in the literature, inflated the gain on six of nine configurations and on two selected operating points 31 and 49 points below the full horizon. Robustness comparisons were the least reliable: two identical training runs differed by a factor of two in VT gain, and an early single-run result showing a 23-point AutoAttack deficit of VT relative to its matched control did not replicate. Only the comparisons tested across seeds should be regarded as established.

5.3 Objectives Revisited

- **O1 (compute-matched suite)** was achieved: every VT point is paired with the fixed horizon of nearest sample-exact SynOps, on separate calibration and test splits (Section 3.7).
- **O2 (models and frontiers)** was achieved for nine configurations: three backbones in both regimes with the fixed-T8 schedule, plus both regimes with the

mixed-T schedule on ResNet-18 (Section 4.4). The mixed-T schedule was not extended to VGG16 and MobileNetV2.

- **O3 (robustness)** was achieved with PGD-20 and AutoAttack through the exit-step logits, for all configurations except the withheld run-1 ResNet-18 measurement (Section 4.7).
- **O4 (gate attacks)** was achieved: wrong-early-exit and compute-overuse attacks were run on every configuration (Section 4.8).
- **O5 (reliability)** was achieved through the VT-neutral rule, the tolerance and stopping-rule analyses, and the three-seed replication on two backbones (Section 4.6, Section 4.9). The MobileNetV2 replication did not complete.

5.4 Summary of Contributions

C1. A compute-matched evaluation protocol for variable timing with sample-exact SynOps accounting and VT-neutral, calibration-split operating-point selection.

C2. A label-preserving compute-overuse attack that turns the exit gate of VT SNNs into an energy-latency attack surface, and evidence that adversarial training largely mitigates it.

C3. Replicated evidence that VT's clean efficiency gain over a compute-matched fixed horizon is real, and evidence that it depends on the training schedule.

C4. A decomposition of VT's robustness cost: equal robustness at equal compute, a consistent AutoAttack deficit relative to the full horizon, and a demonstration that PGD-20 hides it.

C5. A reproducible suite of notebooks, raw results and analysis scripts.

5.5 Implications

For practitioners. VT is worth using when compute matters and inputs vary in difficulty: it gives a better accuracy for a given average budget than a fixed horizon. But a VT deployment should be treated as having an *input-dependent* cost. If the cost budget is a hard constraint, it needs a cap, because an adversary can raise it. If robustness matters, the relevant comparison is with the full horizon, and VT gives up a few points of worst-case robustness in exchange for its savings. A deployment that cannot

afford that should run the full horizon on suspicious inputs or use VT only on clean, trusted data.

For researchers evaluating efficiency methods. The protocol matters as much as the method. Comparing with the full horizon conflates choosing when to stop with simply stopping earlier. Selecting the operating point by the gain one is trying to measure inflates it, and comparing with a fixed-horizon model trained only at the full horizon flatters any early-exit method. The recommendations in Section 4.10 apply beyond VT to any adaptive-compute method for SNNs.

For researchers evaluating SNN robustness. The large PGD–AutoAttack gaps we observed add to the evidence that SNN robustness is easily over-estimated [66]. They also show that the over-estimate can hide relative effects, not just absolute levels: a comparison between two inference modes can come out tied under PGD and clearly different under AutoAttack.

5.6 Limitations

- **Single dataset and horizon.** All results are on CIFAR-10 with $T_{\max} = 8$ and direct encoding.
- **Limited replication.** The multi-seed study covers adversarially trained fixed-T8 ResNet-18 and VGG16 with three seeds each. Clean-trained, mixed-T and MobileNetV2 results rest on single runs, and six checkpoints give limited power to detect small effects.
- **Selection rule for attacks and multi-seed runs.** Attacks and the multi-seed study used max-gain operating points, and the VT-neutral re-selection could be applied only to the clean-data sweeps.
- **Withheld measurement.** The full-test attack results of the first adversarially trained ResNet-18 were inconsistent and are excluded.
- **Compute proxy.** SynOps does not include memory or control costs, and no measurement on neuromorphic hardware was made.
- **Attack coverage.** SNN-specific adaptive attacks and EoT-based attacks on stochastic models were outside scope, and the gate-specific attacks are our own designs and may not be optimal.

- **One VT design.** We studied threshold-based exit with a maximum-softmax score (and four alternatives at matched depth). Learned halting policies, conformal stopping [7] and training objectives designed for early exit were not studied.

5.7 Future Work

Complete the replication. The most direct extension is to replicate the clean-trained and mixed-T configurations across seeds, complete the MobileNetV2 multi-seed run, and re-run the attacks at VT-neutral operating points.

Robust stopping rules. The compute-overuse vulnerability arises because the exit decision depends only on an output-derived confidence that an adversary can lower. Stopping rules whose decisions are robust to small input perturbations, for example certified-confidence or conformal stopping rules [7] or confidence scores computed on adversarially smoothed inputs, could make the gate harder to manipulate. A simple alternative is a learned or calibrated cap on per-input compute.

Training for robust early exit. Our results suggest that robustness accumulates over timesteps. A training objective that rewards robustness at every timestep, in the spirit of temporal efficient training [13] but with adversarial inputs, might let VT exit early without paying the robustness cost. The mixed-T schedule is a first step in this direction, and its interaction with compute-overuse vulnerability deserves further study.

Hardware and event data. Measuring actual energy on neuromorphic hardware, where per-sample early exit may interact with batching and pipelining, would test whether SynOps savings and compute-overuse inflation translate into joules. Event-based datasets with genuine temporal structure, where later timesteps carry new information rather than repeated input, may change both the efficiency gain and the robustness cost.

Stronger attacks. SNN-specific adaptive attacks [66] should be applied to VT models, both to tighten the robustness estimates and to design stronger wrong-early-exit and compute-overuse attacks.

5.8 Concluding Remarks

When we started this work, we expected either to show that variable timing makes spiking networks cheaper at no cost, or to show that it makes them dangerously fragile. The truth is less dramatic and more useful. VT is a genuinely better way to spend a fixed compute budget on clean data. Under attack it is exactly as good as the compute it spends, no better and no worse. Because it spends less, it is less robust than running the full horizon. Its exit gate does not make networks easier to fool, but it does make their energy consumption something an adversary can raise. None of these conclusions would have been visible without compute-matched controls, a selection rule that does not favour the method under test, a strong attack, and more than one training seed. If this thesis has a single lesson beyond its specific findings, it is that the evaluation protocol is part of the result.

References

[1] M. Andriushchenko, F. Croce, N. Flammarion, and M. Hein, "Square Attack: A Query-Efficient Black-Box Adversarial Attack via Random Search," in *European Conference on Computer Vision (ECCV)*, 2020.

[2] A. Athalye, N. Carlini, and D. Wagner, "Obfuscated Gradients Give a False Sense of Security: Circumventing Defenses to Adversarial Examples," in *International Conference on Machine Learning (ICML)*, 2018.

[3] A. Athalye, L. Engstrom, A. Ilyas, and K. Kwok, "Synthesizing Robust Adversarial Examples," in *International Conference on Machine Learning (ICML)*, 2018.

[4] X. Bouthillier *et al.*, "Accounting for Variance in Machine Learning Benchmarks," in *Proceedings of Machine Learning and Systems (MLSys)*, 2021.

[5] N. Carlini *et al.*, *On Evaluating Adversarial Robustness*, 2019. arXiv: 1902.06705.

[6] N. Carlini and D. Wagner, "Towards Evaluating the Robustness of Neural Networks," in *IEEE Symposium on Security and Privacy (S&P)*, 2017, pp. 39–57.

[7] J. Chen, S. Park, and O. Simeone, "Knowing When to Stop: Delay-Adaptive Spiking Neural Network Classifiers With Reliability Guarantees," *IEEE Journal of Selected Topics in Signal Processing*, vol. 19, 2025.

[8] F. Croce *et al.*, "RobustBench: A Standardized Adversarial Robustness Benchmark," in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.

[9] F. Croce and M. Hein, "Minimally Distorted Adversarial Examples with a Fast Adaptive Boundary Attack," in *International Conference on Machine Learning (ICML)*, 2020.

[10] F. Croce and M. Hein, "Reliable Evaluation of Adversarial Robustness with an Ensemble of Diverse Parameter-Free Attacks," in *International Conference on Machine Learning (ICML)*, 2020.

[11] M. Dampfhoffer *et al.*, "Are SNNs Really More Energy-Efficient Than ANNs? An In-Depth Hardware-Aware Study," *IEEE Transactions on Emerging Topics in Computational Intelligence*, 2023.

[12] M. Davies *et al.*, "Loihi: A Neuromorphic Manycore Processor with On-Chip Learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.

[13] S. Deng, Y. Li, S. Zhang, and S. Gu, "Temporal Efficient Training of Spiking Neural Network via Gradient Re-weighting," in *International Conference on Learning Representations (ICLR)*, 2022.

[14] T. DeVries and G. W. Taylor, *Improved Regularization of Convolutional Neural Networks with Cutout*, 2017. arXiv: 1708.04552.

[15] J. Ding, Z. Yu, J. K. Liu, and T. Huang, "Neuromorphic Computing Paradigms Enhance Robustness through Spiking Neural Networks," *Nature Communications*, vol. 16, p. 10 175, 2025.

[16] J. Dodge, S. Gururangan, D. Card, R. Schwartz, and N. A. Smith, "Show Your Work: Improved Reporting of Experimental Results," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[17] W. Fang, Z. Yu, Y. Chen, T. Huang, T. Masquelier, and Y. Tian, "Deep Residual Learning in Spiking Neural Networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[18] W. Gerstner, W. M. Kistler, R. Naud, and L. Paninski, *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition*. Cambridge University Press, 2014.

[19] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and Harnessing Adversarial Examples," in *International Conference on Learning Representations (ICLR)*, 2015.

[20] A. Graves, *Adaptive Computation Time for Recurrent Neural Networks*, 2016. arXiv: 1603.08983.

[21] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," in *International Conference on Machine Learning (ICML)*, 2017.

[22] Y. Han, G. Huang, S. Song, L. Yang, H. Wang, and Y. Wang, "Dynamic Neural Networks: A Survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7436–7456, 2022.

[23] M. Haque, A. Chauhan, C. Liu, and W. Yang, "ILFO: Adversarial Attack on Adaptive Neural Networks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 14 264–14 273.

[24] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[25] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep Reinforcement Learning That Matters," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

[26] D. Hendrycks and K. Gimpel, "A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks," in *International Conference on Learning Representations (ICLR)*, 2017.

[27] S. Hong, Y. Kaya, I.-V. Modoranu, and T. Dumitras, "A Panda? No, It's a Sloth: Slowdown Attacks on Adaptive Multi-Exit Neural Network Inference," in *International Conference on Learning Representations (ICLR)*, 2021.

[28] M. Horowitz, "1.1 Computing's Energy Problem (and What We Can Do About It)," in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2014, pp. 10–14.

[29] G. Huang, D. Chen, T. Li, F. Wu, L. van der Maaten, and K. Q. Weinberger, "Multi-Scale Dense Networks for Resource Efficient Image Classification," in *International Conference on Learning Representations (ICLR)*, 2018.

[30] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *International Conference on Machine Learning (ICML)*, 2015.

[31] Y. Kaya, S. Hong, and T. Dumitras, "Shallow-Deep Networks: Understanding and Mitigating Network Overthinking," in *International Conference on Machine Learning (ICML)*, 2019.

[32] H. Kim, *Torchattacks: A PyTorch Repository for Adversarial Attacks*, 2020. arXiv: 2010.01950.

[33] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," University of Toronto, Tech. Rep., 2009.

[34] S. Kundu, M. Pedram, and P. A. Beerel, "HIRE-SNN: Harnessing the Inherent Robustness of Energy-Efficient Deep Spiking Neural Networks by Training with Crafted Input Noise," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[35] S. Laskaridis, A. Kouris, and N. D. Lane, "Adaptive Inference through Early-Exit Networks: Design, Challenges and Directions," in *Proceedings of the 5th International Workshop on Embedded and Mobile Deep Learning (EMDL)*, 2021.

[36] C. Li, E. G. Jones, and S. Furber, "Unleashing the Potential of Spiking Neural Networks with Dynamic Confidence," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[37] Y. Li, A. Moitra, T. Geller, and P. Panda, "Input-Aware Dynamic Timestep Spiking Neural Networks for Efficient In-Memory Computing," in *ACM/IEEE Design Automation Conference (DAC)*, 2023.

[38] L. Liang, K. Xu, X. Hu, L. Deng, and Y. Xie, "Toward Robust Spiking Neural Network Against Adversarial Perturbation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[39] Z. C. Lipton and J. Steinhardt, "Troubling Trends in Machine Learning Scholarship," *Queue*, vol. 17, no. 1, 2019.

[40] I. Loshchilov and F. Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts," in *International Conference on Learning Representations (ICLR)*, 2017.

[41] W. Maass, "Networks of Spiking Neurons: The Third Generation of Neural Network Models," *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.

[42] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards Deep Learning Models Resistant to Adversarial Attacks," in *International Conference on Learning Representations (ICLR)*, 2018.

[43] G. Melis, C. Dyer, and P. Blunsom, "On the State of the Art of Evaluation in Neural Language Models," in *International Conference on Learning Representations (ICLR)*, 2018.

[44] P. A. Merolla *et al.*, "A Million Spiking-Neuron Integrated Circuit with a Scalable Communication Network and Interface," *Science*, vol. 345, no. 6197, pp. 668–673, 2014.

[45] P. Micikevicius *et al.*, "Mixed Precision Training," in *International Conference on Learning Representations (ICLR)*, 2018.

[46] B. Na, J. Mok, S. Park, D. Lee, H. Choe, and S. Yoon, "AutoSNN: Towards Energy-Efficient Spiking Neural Networks," in *International Conference on Machine Learning (ICML)*, 2022.

[47] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate Gradient Learning in Spiking Neural Networks: Bringing the Power of Gradient-Based Optimization to Spik-

ing Neural Networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.

[48] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[49] D. Picard, *Torch.manual_seed(3407) Is All You Need: On the Influence of Random Seeds in Deep Learning Architectures for Computer Vision*, 2021. arXiv: 2109.08203.

[50] Rao *et al.*, "VT-SNN: Variable Time-Step Spiking Neural Network Based on Uncertainty Measure and Its Application in Brain Disease Diagnosis," in *International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2025.

[51] L. Rice, E. Wong, and J. Z. Kolter, "Overfitting in Adversarially Robust Deep Learning," in *International Conference on Machine Learning (ICML)*, 2020.

[52] K. Roy, A. Jaiswal, and P. Panda, "Towards Spike-Based Machine Intelligence with Neuromorphic Computing," *Nature*, vol. 575, no. 7784, pp. 607–617, 2019.

[53] B. Rueckauer, I.-A. Lungu, Y. Hu, M. Pfeiffer, and S.-C. Liu, "Conversion of Continuous-Valued Deep Networks to Efficient Event-Driven Networks for Image Classification," *Frontiers in Neuroscience*, vol. 11, p. 682, 2017.

[54] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[55] M. Sensoy, L. Kaplan, and M. Kandemir, "Evidential Deep Learning to Quantify Classification Uncertainty," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[56] S. Sharmin, P. Panda, S. S. Sarwar, C. Lee, W. Ponghiran, and K. Roy, "A Comprehensive Analysis on Adversarial Robustness of Spiking Neural Networks," in *International Joint Conference on Neural Networks (IJCNN)*, 2019.

[57] S. Sharmin, N. Rathi, P. Panda, and K. Roy, "Inherent Adversarial Robustness of Deep Spiking Neural Networks: Effects of Discrete Input Encoding and Non-Linear Activations," in *European Conference on Computer Vision (ECCV)*, 2020.

[58] I. Shumailov, Y. Zhao, D. Bates, N. Papernot, R. Mullins, and R. Anderson, "Sponge Examples: Energy-Latency Attacks on Neural Networks," in *IEEE European Symposium on Security and Privacy (EuroS&P)*, 2021.

[59] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," in *International Conference on Learning Representations (ICLR)*, 2015.

[60] M. Sorbaro, Q. Liu, M. Bortone, and S. Sheik, "Optimizing the Energy Consumption of Spiking Neural Networks for Neuromorphic Applications," *Frontiers in Neuroscience*, vol. 14, p. 662, 2020.

[61] C. Szegedy, W. Zaremba, I. Sutskever, *et al.*, "Intriguing Properties of Neural Networks," in *International Conference on Learning Representations (ICLR)*, 2014.

[62] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks," in *International Conference on Pattern Recognition (ICPR)*, 2016, pp. 2464–2469.

[63] F. Tramèr, N. Carlini, W. Brendel, and A. Madry, "On Adaptive Attacks to Adversarial Example Defenses," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[64] F. Tramèr, A. Kurakin, N. Papernot, I. Goodfellow, D. Boneh, and P. McDaniel, "Ensemble Adversarial Training: Attacks and Defenses," in *International Conference on Learning Representations (ICLR)*, 2018.

[65] D. Tsipras, S. Santurkar, L. Engstrom, A. Turner, and A. Madry, "Robustness May Be at Odds with Accuracy," in *International Conference on Learning Representations (ICLR)*, 2019.

[66] J. Wang, D. Zhao, R. Chen, Q. Zhang, and Y. Zeng, "Towards Reliable Evaluation of Adversarial Robustness for Spiking Neural Networks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.

[67] J. Wang, D. Zhao, C. Du, X. He, Q. Zhang, and Y. Zeng, "Random Heterogeneous Spiking Neural Network for Adversarial Defense," *iScience*, vol. 28, no. 6, 2025.

[68] E. Wong, L. Rice, and J. Z. Kolter, "Fast Is Better Than Free: Revisiting Adversarial Training," in *International Conference on Learning Representations (ICLR)*, 2020.

[69] K. Wu, M. Yao, Y. Chou, *et al.*, "RSC-SNN: Exploring the Trade-off Between Adversarial Robustness and Accuracy in Spiking Neural Networks via Randomized Smoothing Coding," in *ACM International Conference on Multimedia (ACM MM)*, 2024.

[70] Y. Wu, L. Deng, G. Li, J. Zhu, and L. Shi, "Spatio-Temporal Backpropagation for Training High-Performance Spiking Neural Networks," *Frontiers in Neuroscience*, vol. 12, p. 331, 2018.

[71] H. Zhang, Y. Yu, J. Jiao, E. Xing, L. El Ghaoui, and M. Jordan, "Theoretically Principled Trade-off between Robustness and Accuracy," in *International Conference on Machine Learning (ICML)*, 2019.

[72] H. Zheng, Y. Wu, L. Deng, Y. Hu, and G. Li, "Going Deeper With Directly-Trained Larger Spiking Neural Networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.

Appendices

Appendix A

Hyperparameters

Table A.1 lists every setting used to train and evaluate the models in this thesis. Settings not listed are the defaults of the code base and are recorded in the notebooks.

Table A.1: Training, model and evaluation hyperparameters.

Setting	Value
Model	
Neuron	Deterministic LIF, $\tau = 2$, $V_{\text{th}} = 0.5$, hard reset, detached reset
Surrogate gradient	$\alpha/(2(1+\alpha\lvert x\rvert)^2)$, $\alpha = 4$
Encoding	Direct (image repeated at every timestep), per-channel normalisation
Maximum horizon	$T_{\max} = 8$
Stochastic / Parseval / weight perturbation	Disabled
Training	
Epochs	100
Optimiser	SGD, momentum 0.9, weight decay 10^{-4}
Learning rate	0.1, 5-epoch linear warm-up, cosine decay
Batch size	256
Precision	Automatic mixed precision (float16), gradient clipping at norm 1.0
Augmentation	Random crop (padding 4), horizontal flip, Cutout (1×16 px)
Clean regime	100% clean images
Adversarial regime	100% RFGSM images, $\epsilon = \alpha = 8/255$, random start $\pm 0.1\epsilon$
Timestep schedule	Fixed-T8: $T = 8$; mixed-T: $T \sim \mathcal{U}\{1, \dots, 8\}$ per batch
Checkpoint selection	Best clean validation accuracy at $T = 8$, evaluated every 5 epochs
Seeds	0 (single-run models); 0, 1, 2 (multi-seed)
Evaluation	
Splits	40,000 train / 5,000 validation / 5,000 calibration / 10,000 test
Fixed horizons	$T \in \{1, \dots, 8\}$
VT thresholds	$\theta \in \{0.5, 0.6, 0.7, 0.8, 0.85, 0.9, 0.95, 0.99\}$
Compute matching	Nearest mean SynOps, sample-exact VT accounting
VT-neutral tolerance	$\delta = 1$ (primary); 0.5 and 2 (sensitivity)
Temperature (temperature softmax)	2
Attacks	
Budget	ℓ_∞, $\epsilon = 8/255$
PGD-20	20 steps, step size $2/255$, uniform random start, CE loss on exit-step logits
AutoAttack	Standard version (APGD-CE, APGD-T, FAB-T, Square), attack seed 0
Exit handling	Full 8-step trace; loss on accumulated logits at the deployed exit step
Wrong early exit	Eq. (3.13), confidence weight $\lambda_c = 0.25$
Compute overuse	Eq. (3.14), label-preservation weight $\lambda_p = 1$
Attack subset	Class-balanced; 2,000 (single run), 1,000 (multi-seed), seed 0

Appendix B

Additional Results

This appendix collects the full result tables from which the figures and summary tables of Chapter 4 are drawn. All tables are generated from the raw result files by `analysis/make_tables.py`.

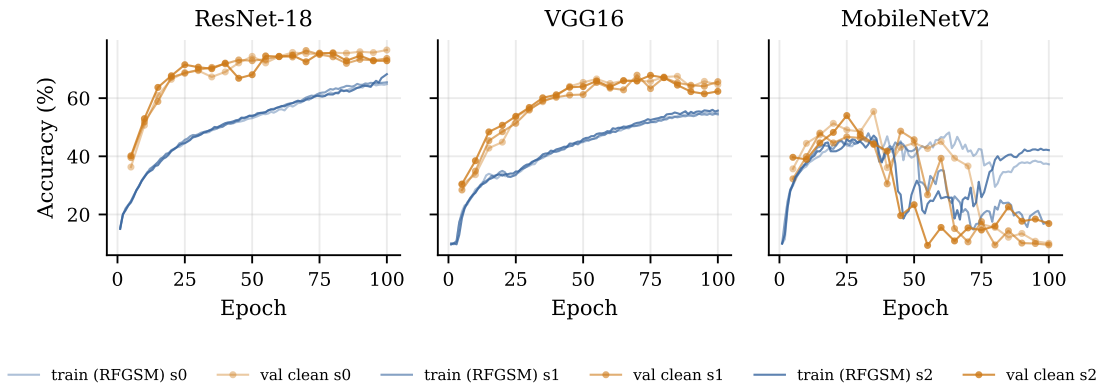


Figure B.1: Training curves of the multi-seed adversarially trained models: RFGSM training accuracy (blue, every epoch) and clean validation accuracy at $T = 8$ (orange, every five epochs) for seeds 0–2. MobileNetV2 training is unstable, and one seed's training accuracy collapses after about epoch 40.

Table B.13: All single-run attack results, including the neighbouring thresholds of each attacked VT operating point. The run-1 adversarial ResNet-18 is omitted (see Section 4.11).

Configuration	Mode	θ	Role	Attack	Objective	Acc.	Depth	n
R18 clean	fixed	—	fixed T8 baseline	pgd	untargeted	0.00	8.00	10000
R18 clean	vt	0.60	lower neighbor	pgd	untargeted	0.00	2.00	10000
R18 clean	vt	0.70	selected	pgd	untargeted	0.02	2.00	10000
R18 clean	vt	0.80	upper neighbor	pgd	untargeted	0.00	2.01	10000
R18 clean	vt	0.60	lower neighbor	pgd	wrong early exit	0.00	2.00	10000

Configuration	Mode	θ	Role	Attack	Objective	Acc.	Depth	n
R18 clean	vt	0.70	selected	pgd	wrong early exit	0.00	2.00	10000
R18 clean	vt	0.80	upper neighbor	pgd	wrong early exit	0.00	2.00	10000
R18 clean	vt	0.60	lower neighbor	pgd	compute overuse	59.74	3.09	10000
R18 clean	vt	0.70	selected	pgd	compute overuse	60.58	3.58	10000
R18 clean	vt	0.80	upper neighbor	pgd	compute overuse	63.10	4.23	10000
R18 clean	fixed	—	fixed T8 baseline	autoattack	untargeted	0.06	8.00	10000
R18 clean	vt	0.70	selected	autoattack	untargeted	0.02	2.14	10000
R18 adv (2)	fixed	—	fixed T8 baseline	pgd	untargeted	46.10	8.00	2000
R18 adv (2)	vt	0.50	lower neighbor	pgd	untargeted	47.75	5.13	2000
R18 adv (2)	vt	0.60	selected	pgd	untargeted	48.70	5.86	2000
R18 adv (2)	vt	0.70	upper neighbor	pgd	untargeted	46.35	6.43	2000
R18 adv (2)	vt	0.50	lower neighbor	pgd	wrong early exit	45.80	4.75	2000
R18 adv (2)	vt	0.60	selected	pgd	wrong early exit	45.95	5.62	2000
R18 adv (2)	vt	0.70	upper neighbor	pgd	wrong early exit	47.75	6.31	2000
R18 adv (2)	vt	0.50	lower neighbor	pgd	compute overuse	88.90	4.84	2000
R18 adv (2)	vt	0.60	selected	pgd	compute overuse	92.00	5.40	2000
R18 adv (2)	vt	0.70	upper neighbor	pgd	compute overuse	93.25	6.04	2000
R18 adv (2)	fixed	—	fixed T8 baseline	autoattack	untargeted	28.85	8.00	2000
R18 adv (2)	vt	0.60	selected	autoattack	untargeted	24.15	5.55	2000
R18 clean mixT	fixed	—	fixed T8 baseline	pgd	untargeted	0.00	8.00	2000
R18 clean mixT	vt	0.50	lower neighbor	pgd	untargeted	0.15	1.01	2000
R18 clean mixT	vt	0.60	selected	pgd	untargeted	0.15	1.02	2000
R18 clean mixT	vt	0.70	upper neighbor	pgd	untargeted	0.10	1.05	2000
R18 clean mixT	vt	0.50	lower neighbor	pgd	wrong early exit	3.35	1.10	2000
R18 clean mixT	vt	0.60	selected	pgd	wrong early exit	2.60	1.18	2000
R18 clean mixT	vt	0.70	upper neighbor	pgd	wrong early exit	2.10	1.25	2000
R18 clean mixT	vt	0.50	lower neighbor	pgd	compute overuse	85.40	1.21	2000
R18 clean mixT	vt	0.60	selected	pgd	compute overuse	81.60	1.55	2000
R18 clean mixT	vt	0.70	upper neighbor	pgd	compute overuse	75.30	2.08	2000
R18 clean mixT	fixed	—	fixed T8 baseline	autoattack	untargeted	0.10	8.00	2000
R18 clean mixT	vt	0.60	selected	autoattack	untargeted	2.25	1.13	2000
R18 adv mixT	fixed	—	fixed T8 baseline	pgd	untargeted	40.85	8.00	2000
R18 adv mixT	vt	0.70	lower neighbor	pgd	untargeted	40.90	5.59	2000
R18 adv mixT	vt	0.80	selected	pgd	untargeted	41.50	6.35	2000
R18 adv mixT	vt	0.85	upper neighbor	pgd	untargeted	40.80	6.77	2000
R18 adv mixT	vt	0.70	lower neighbor	pgd	wrong early exit	42.00	5.35	2000
R18 adv mixT	vt	0.80	selected	pgd	wrong early exit	42.55	6.27	2000
R18 adv mixT	vt	0.85	upper neighbor	pgd	wrong early exit	41.80	6.66	2000
R18 adv mixT	vt	0.70	lower neighbor	pgd	compute overuse	93.80	5.30	2000
R18 adv mixT	vt	0.80	selected	pgd	compute overuse	94.95	6.08	2000
R18 adv mixT	vt	0.85	upper neighbor	pgd	compute overuse	95.70	6.54	2000
R18 adv mixT	fixed	—	fixed T8 baseline	autoattack	untargeted	29.45	8.00	2000

Configuration	Mode	θ	Role	Attack	Objective	Acc.	Depth	n
R18 adv mixT	vt	0.80	selected	autoattack	untargeted	25.55	5.98	2000
VGG clean	fixed	—	fixed T8 baseline	pgd	untargeted	0.05	8.00	2000
VGG clean	vt	0.60	lower neighbor	pgd	untargeted	0.10	2.32	2000
VGG clean	vt	0.70	selected	pgd	untargeted	0.10	2.40	2000
VGG clean	vt	0.80	upper neighbor	pgd	untargeted	0.05	2.52	2000
VGG clean	vt	0.60	lower neighbor	pgd	wrong early exit	0.10	2.32	2000
VGG clean	vt	0.70	selected	pgd	wrong early exit	0.00	2.45	2000
VGG clean	vt	0.80	upper neighbor	pgd	wrong early exit	0.10	2.62	2000
VGG clean	vt	0.60	lower neighbor	pgd	compute overuse	69.80	3.99	2000
VGG clean	vt	0.70	selected	pgd	compute overuse	72.10	4.70	2000
VGG clean	vt	0.80	upper neighbor	pgd	compute overuse	75.40	5.41	2000
VGG clean	fixed	—	fixed T8 baseline	autoattack	untargeted	0.00	8.00	2000
VGG clean	vt	0.70	selected	autoattack	untargeted	0.00	2.82	2000
VGG adv	fixed	—	fixed T8 baseline	pgd	untargeted	45.90	8.00	2000
VGG adv	vt	0.50	lower neighbor	pgd	untargeted	44.95	6.17	2000
VGG adv	vt	0.60	selected	pgd	untargeted	44.45	6.69	2000
VGG adv	vt	0.70	upper neighbor	pgd	untargeted	44.75	7.13	2000
VGG adv	vt	0.50	lower neighbor	pgd	wrong early exit	45.40	5.98	2000
VGG adv	vt	0.60	selected	pgd	wrong early exit	46.35	6.62	2000
VGG adv	vt	0.70	upper neighbor	pgd	wrong early exit	46.70	7.05	2000
VGG adv	vt	0.50	lower neighbor	pgd	compute overuse	79.25	5.83	2000
VGG adv	vt	0.60	selected	pgd	compute overuse	80.95	6.31	2000
VGG adv	vt	0.70	upper neighbor	pgd	compute overuse	81.45	6.80	2000
VGG adv	fixed	—	fixed T8 baseline	autoattack	untargeted	10.75	8.00	2000
VGG adv	vt	0.60	selected	autoattack	untargeted	8.55	6.60	2000
MBv2 clean	fixed	—	fixed T8 baseline	pgd	untargeted	5.10	8.00	2000
MBv2 clean	vt	0.90	lower neighbor	pgd	untargeted	4.30	6.55	2000
MBv2 clean	vt	0.95	selected	pgd	untargeted	4.95	7.17	2000
MBv2 clean	vt	0.99	upper neighbor	pgd	untargeted	4.85	7.86	2000
MBv2 clean	vt	0.90	lower neighbor	pgd	wrong early exit	4.20	6.22	2000
MBv2 clean	vt	0.95	selected	pgd	wrong early exit	4.05	6.95	2000
MBv2 clean	vt	0.99	upper neighbor	pgd	wrong early exit	3.80	7.82	2000
MBv2 clean	vt	0.90	lower neighbor	pgd	compute overuse	97.05	7.38	2000
MBv2 clean	vt	0.95	selected	pgd	compute overuse	97.20	7.71	2000
MBv2 clean	vt	0.99	upper neighbor	pgd	compute overuse	97.10	7.99	2000
MBv2 clean	fixed	—	fixed T8 baseline	autoattack	untargeted	5.25	8.00	2000
MBv2 clean	vt	0.95	selected	autoattack	untargeted	5.40	7.23	2000
MBv2 adv	fixed	—	fixed T8 baseline	pgd	untargeted	35.60	8.00	2000
MBv2 adv	vt	0.50	lower neighbor	pgd	untargeted	34.55	6.85	2000
MBv2 adv	vt	0.60	selected	pgd	untargeted	34.75	7.45	2000
MBv2 adv	vt	0.70	upper neighbor	pgd	untargeted	35.80	7.79	2000
MBv2 adv	vt	0.50	lower neighbor	pgd	wrong early exit	34.50	6.69	2000

Configuration	Mode	θ	Role	Attack	Objective	Acc.	Depth	n
MBv2 adv	vt	0.60	selected	pgd	wrong early exit	33.35	7.42	2000
MBv2 adv	vt	0.70	upper neighbor	pgd	wrong early exit	32.90	7.78	2000
MBv2 adv	vt	0.50	lower neighbor	pgd	compute overuse	64.40	6.59	2000
MBv2 adv	vt	0.60	selected	pgd	compute overuse	67.10	7.26	2000
MBv2 adv	vt	0.70	upper neighbor	pgd	compute overuse	66.70	7.68	2000
MBv2 adv	fixed	—	fixed T8 baseline	autoattack	untargeted	27.60	8.00	2000
MBv2 adv	vt	0.60	selected	autoattack	untargeted	27.55	7.39	2000

Table B.1: Mean SynOps per image (10^6) of fixed-timestep inference at each horizon (test set).

Configuration	T=1	2	3	4	5	6	7	8
R18 clean	17.1	78.7	127.0	191.8	237.3	304.7	349.5	413.7
R18 adv (1)	22.1	65.2	112.0	156.9	196.1	246.9	285.7	333.3
R18 adv (2)	10.6	56.5	89.9	138.6	164.1	218.9	240.9	292.6
R18 clean mixT	42.2	122.0	191.3	270.0	338.9	418.9	487.0	565.0
R18 adv mixT	33.2	90.9	144.4	200.7	254.2	312.2	364.4	420.4
VGG clean	4.3	28.5	51.3	81.4	100.9	132.5	151.4	182.3
VGG adv	4.5	20.4	38.1	59.0	73.9	96.5	110.9	133.3
MBv2 clean	45.3	92.3	139.4	187.7	234.0	282.6	328.7	377.0
MBv2 adv	45.9	96.2	146.4	196.9	245.4	296.7	345.0	395.8

Table B.2: VT sweep for R18 clean (test set, max-softmax rule).

θ	Accuracy (%)	Mean depth	SynOps (10^6)
0.50	86.53	2.12	84.7
0.60	87.72	2.24	90.9
0.70	88.91	2.39	99.4
0.80	89.80	2.62	111.4
0.85	90.13	2.79	120.7
0.90	90.62	3.01	133.0
0.95	90.96	3.37	152.6
0.99	91.26	4.22	199.4

Table B.3: VT sweep for R18 adv (1) (test set, max-softmax rule).

θ	Accuracy (%)	Mean depth	SynOps (10^6)
0.50	71.48	4.33	163.3
0.60	72.83	5.01	193.3
0.70	73.49	5.58	219.2
0.80	73.66	6.11	243.4
0.85	73.69	6.40	256.4
0.90	73.73	6.71	271.2
0.95	73.73	7.10	289.2
0.99	73.73	7.61	314.0

Table B.4: VT sweep for R18 adv (2) (test set, max-softmax rule).

θ	Accuracy (%)	Mean depth	SynOps (10^6)
0.50	75.15	4.54	151.6
0.60	76.42	5.13	174.7
0.70	76.83	5.62	194.2
0.80	77.13	6.14	215.2
0.85	77.16	6.41	226.0
0.90	77.20	6.73	239.0
0.95	77.19	7.14	255.9
0.99	77.19	7.69	279.0

Table B.5: VT sweep for R18 clean mixT (test set, max-softmax rule).

θ	Accuracy (%)	Mean depth	SynOps (10^6)
0.50	52.47	1.20	58.1
0.60	59.63	1.38	72.0
0.70	66.26	1.63	91.0
0.80	72.90	1.98	117.4
0.85	76.39	2.23	136.1
0.90	80.03	2.57	161.1
0.95	84.15	3.12	201.7
0.99	88.92	4.24	284.8

Table B.6: VT sweep for R18 adv mixT (test set, max-softmax rule).

θ	Accuracy (%)	Mean depth	SynOps (10^6)
0.50	70.60	3.16	146.8
0.60	74.70	3.98	191.1
0.70	76.47	4.73	231.8
0.80	77.50	5.43	270.7
0.85	77.63	5.78	290.3
0.90	77.74	6.18	313.0
0.95	77.75	6.70	342.9
0.99	77.75	7.42	385.1

Table B.7: VT sweep for VGG clean (test set, max-softmax rule).

θ	Accuracy (%)	Mean depth	SynOps (10^6)
0.50	84.16	2.89	49.2
0.60	85.78	3.20	57.1
0.70	86.99	3.53	65.2
0.80	87.75	3.89	74.1
0.85	87.94	4.08	78.9
0.90	88.08	4.33	85.2
0.95	88.14	4.68	94.2
0.99	88.18	5.46	114.3

Table B.8: VT sweep for VGG adv (test set, max-softmax rule).

θ	Accuracy (%)	Mean depth	SynOps (10^6)
0.50	64.64	5.74	86.6
0.60	65.21	6.27	97.0
0.70	65.41	6.69	105.3
0.80	65.49	7.05	112.6
0.85	65.48	7.21	115.9
0.90	65.49	7.39	119.6
0.95	65.49	7.60	124.2
0.99	65.49	7.87	130.2

Table B.9: VT sweep for MBv2 clean (test set, max-softmax rule).

θ	Accuracy (%)	Mean depth	SynOps (10^6)
0.50	28.96	2.38	110.4
0.60	39.32	2.95	137.6
0.70	49.96	3.67	171.7
0.80	60.53	4.61	216.0
0.85	66.50	5.17	242.9
0.90	71.33	5.80	272.7
0.95	75.68	6.56	308.6
0.99	78.08	7.41	349.2

Table B.10: VT sweep for MBv2 adv (test set, max-softmax rule).

θ	Accuracy (%)	Mean depth	SynOps (10^6)
0.50	49.65	6.55	323.2
0.60	50.92	7.21	356.2
0.70	51.38	7.62	377.0
0.80	51.48	7.86	388.9
0.85	51.49	7.93	392.3
0.90	51.49	7.97	394.5
0.95	51.49	8.00	395.6
0.99	51.49	8.00	395.8

Table B.11: Compute at the VT-neutral operating points ($\delta = 1$, test set). Mismatch: VT SynOps relative to the matched control. CI: binomial 95% half-width of the VT accuracy.

Configuration	θ	VT SynOps	T^*	Matched SynOps	Mismatch	T=8 SynOps	CI
R18 clean	0.90	133.0	3	127.0	+4.7%	413.7	±0.57
R18 adv (1)	0.60	193.3	5	196.1	-1.5%	333.3	±0.87
R18 adv (2)	0.60	174.7	5	164.1	+6.5%	292.6	±0.83
R18 clean mixT	0.99	284.8	4	270.0	+5.5%	565.0	±0.62
R18 adv mixT	0.80	270.7	5	254.2	+6.5%	420.4	±0.82
VGG clean	0.80	74.1	4	81.4	-8.9%	182.3	±0.64
VGG adv	0.50	86.6	6	96.5	-10.2%	133.3	±0.94
MBv2 clean	0.99	349.2	7	328.7	+6.2%	377.0	±0.81
MBv2 adv	0.60	356.2	7	345.0	+3.2%	395.8	±0.98

Table B.12: Stopping-rule results for the two configurations whose reference depth came from a degenerate max-gain operating point (test set).

Configuration	Rule	Accuracy (%)	Mean depth
R18 clean mixT	fixed T8 baseline	90.57	8.00
R18 clean mixT	max softmax	59.63	1.38
R18 clean mixT	margin	65.36	1.62
R18 clean mixT	entropy	55.87	1.26
R18 clean mixT	temperature softmax	72.24	1.84
R18 clean mixT	evidential surrogate	45.15	1.05
MBv2 clean	fixed T8 baseline	77.66	8.00
MBv2 clean	max softmax	28.96	2.38
MBv2 clean	margin	48.82	3.58
MBv2 clean	entropy	29.92	2.45
MBv2 clean	temperature softmax	57.05	4.29
MBv2 clean	evidential surrogate	18.10	2.00

Table B.14: Attack-seed variance: robust accuracy (%, mean $\pm$ SD over three attack seeds) on one adversarially trained checkpoint per backbone ($n = 1{,}000$).

	Full T=8		Matched		VT	
Backbone	PGD	AA	PGD	AA	PGD	AA
ResNet-18	56.8 ± 0.70	32.0 ± 0.31	55.1 ± 1.14	23.9 ± 0.44	56.3 ± 1.33	23.2 ± 1.74
VGG16	45.0 ± 1.00	18.4 ± 0.12	43.8 ± 0.26	20.4 ± 0.45	45.2 ± 0.55	16.0 ± 0.81
MobileNetV2	45.5 ± 0.78	20.6 ± 0.46	41.2 ± 1.04	13.4 ± 0.66	44.4 ± 0.85	10.8 ± 0.40

Table B.15: Multi-seed training summaries (adversarial fixed-T8 regime). Train adv.: RFGSM training accuracy in the final epoch. Best val.: best clean validation accuracy at $T = 8$ (evaluated every five epochs).

Backbone	Seed	Epochs	Train adv. (%)	Best val. (%)
ResNet-18	0	100	64.81	76.56
ResNet-18	1	100	65.47	76.38
ResNet-18	2	100	68.25	75.48
VGG16	0	100	55.00	67.52
VGG16	1	100	54.42	67.90
VGG16	2	100	55.72	67.86
MobileNetV2	0	100	37.26	55.46
MobileNetV2	1	100	17.49	48.66
MobileNetV2	2	100	42.11	54.02

Appendix C

Reproducibility

C.1 Repository Layout

The experimental code and all raw results are kept in a single repository.

- `model.py`, `stopping.py`, `metrics.py`, `attacks.py`, `training.py`, `experiments.py`, `data.py`, `configs.py`: the experimental suite (models and LIF neurons, stopping rules, SynOps monitor, attacks, training loop, evaluation protocol, data splits, configuration dataclasses).
- `build_vanilla_{resnet18,vgg16,mobilenetv2}_kaggle_notebook.py`: builders for the self-contained single-run notebooks. The ResNet-18 builder emits four notebooks, one for each regime–schedule combination.
- `build_multiseed_kaggle_notebook.py`: builder for the multi-seed notebooks (one per backbone), including training of every seed.
- `Findings/`: one folder of raw CSV results per configuration (calibration and test sweeps, matched controls, selected thresholds, stopping rules, attack results) and per multi-seed run (per-checkpoint results, attack-seed results, training histories).
- `thesis report/vt_thesis/analysis/`: `build_facts.py` (computes every number in the thesis and writes `facts.json` and `facts.md`), `make_figures.py` (all data figures and the pooled statistics) and `make_tables.py` (appendix tables).

C.2 Regenerating the Results

Training and evaluation. Each notebook is generated by running its builder script locally and uploaded to Kaggle with a GPU accelerator. With internet disabled, CIFAR-10 and the AutoAttack and torchattacks packages must be attached as Kaggle datasets, and the notebooks locate them automatically. Training regime and schedule are selected by the `TRAIN_REGIME` and `TRAIN_SCHEDULE` variables at the top of the configuration cell. Each notebook writes its CSV results and checkpoint to the Kaggle working directory.

Analysis. With the results unpacked into `Findings/`, the analysis is regenerated from the thesis directory with

```
python analysis/build_facts.py
python analysis/make_figures.py
python analysis/make_tables.py
```

No GPU is needed for the analysis: operating-point re-selection, compute matching and all statistics are computed from the saved sweeps.

Document. The thesis is compiled with Tectonic (XeTeX engine) and Biber:

```
tectonic -X compile main.tex
```

The Biber version must match the biblatex version bundled with Tectonic (Biber 2.17 for Tectonic 0.17).

C.3 Determinism

All models use deterministic neurons, and data splits, attack subsets and training seeds are fixed. Results may still differ in the last digit between runs because of non-deterministic cuDNN kernels, TF32 arithmetic and mixed precision. The multi-seed experiments quantify run-to-run variance at the level that matters for our conclusions.